

Programsko inženjerstvo ak. god. 2025./2026.

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

Dish Detective

Tim: <TG10.4>

Ime tima: Dish Detectors

Članovi:

- Luka Ražem - voditelj
- Andrea Marić
- Matea Maračić
- Loren Čekada
- Matej Božac
- Ivan Marfat

Nastavnik: Vlado Sruk

1. Opis problema

Mnogi se studenti svakodnevno suočavaju s problemom dugog čekanja u redu za menzu, a razočaranje je još veće kada napokon dođu na red i tamo ih dočeka ponuda koja uopće nije navedena na dnevnom jelovniku. Projekt Dish Detektiv osmišljen je kako bi riješio ovaj problem i studentima olakšao svakodnevni život. Dish Detective je web aplikacija koja studentima pruža informacije o trenutno dostupnim jelima u pojedinim menzama te ih obavještava kada su njihova omiljena jela na meniju. Naša je aplikacija napravljena u interesu studenata i njihovih potreba te kako bi umanjili stres i nezadovoljstvo koje mnogi studenti povezuju sa studentskim restoranima.

2. Cilj projektnog zadatka

Glavni cilj projekta Dish Detective je razviti platformu koja omogućava studentima pregled trenutnog stanja hrane u menzi, a radnicima menze omogućuje da vrlo jednostavno i brzo ažuriraju stanje njihove menze. Nadalje, cilj je omogućiti studentima da preko aplikacije dobe obavijest u trenutku kada hrana za koju su zainteresirani postane dostupna.

3. Problematika zadatka

Aplikacija rješava problem koji često muči studente, iako imaju popis svih jela koja će se posluživati u nekom danu, ne znaju koja su jela dostupna u nekom trenutku bez da fizički posjete menzu. Nadalje, studente često ne zanimaju sva jela, već samo nekoliko određenih, te žele biti obaviješteni kada ta jela postanu dostupna kako bi znali kada otići u menzu. Studente također zanima koliko je vremena prošlo od kad je neko jelo postalo dostupno ili nedostupno. S druge strane radnici u menzi rade u vrlo dinamičnom okruženju te žele potrošiti što manje vremena na ažuriranje stanja menze.

4. Potencijalna korist projekta

Projekt Dish Detective pruža veliku korist studentima, prvenstveno kroz unapređenje planiranja svakodnevnih aktivnosti i produktivno upravljanje s vremenom. Aplikacija omogućuje studentima da u svakom trenutku imaju uvid u trenutčnu ponudu u menzi kako bi izbjegli odlaske u menze koje ne poslužuju obroke prema njihovoj volji. Na ovaj način, korisnici bolje upravljaju s vremenom te izbjegavanju bespotrebne frustracije za vrijeme obroka.

5. Postojeća slična rješenja

Na tržištu trenutčno nema rješenja sličnog Dish Detectiveu što predstavlja rupu u tržištu koju želimo popuniti našim rješenjem.

6. Skup korisnika zainteresiranih za rješenje

Aplikacija Dish Detective primarno je osmišljena s ciljem olakšavanja svakodnevnog života studenata koji se redovito koriste uslugama sveučilišnih restorana te menza u studentskim domovima. Studenti predstavljaju primarnu skupinu korisnika, s obzirom na to da im rješenje omogućuje jednostavniji

pristup informacijama o ponudi i bolju organizaciju vremena.

Uz studente, važnu skupinu čine i zaposlenici menza, čiji je interes poboljšanje organizacije rada i planiranja pripreme obroka, kao i općenitog iskustva svojih posjetitelja.

7. Mogućnost prilagodbe rješenja

Postoji velik broj mogućih prilagodbi rješenja, na primjer da se proširi na ostale gradove u Hrvatskoj kako bi uključili što više studentskih restorana. Uz povratne informacije korisnika, moguće je bez temeljnih promjena aplikacije dodati još funkcionalnosti kako bi omogućili što bolje korisničko iskustvo.

8. Opseg projektnog zadatka

Opseg projekta uključuje:

- Pregled trenutnog stanja menza
 - Kreiranje korisničkih računa (student)
 - Odabir različitih uloga unutar aplikacije: administrator, voditelj menze, radnik u menzi, student
 - Sustav notifikacije studenata kada se stanje menze promjeni
 - Kreacija dnevnog menija (voditelj menze)
 - Kreacija računa za voditelje menza i radnika u menzi (administrator)
 - Pregled vremena početka dostupnosti pojedinog jela te koliko dugo neko jelo nije dostupno
 - Mogućnost ocjenjivanja aktivnih jela
 - Mogućnost glasanja na anketi za koje jelo dodati, a koje maknuti iz menija
 - Mogućnost pronalaženja menzi koje su geografski blizu korisniku
-

9. Moguće nadogradnje projektnog zadatka

Projekt je moguće nadograditi na puno načina, a neki od njih su:

- proširenje na ostale gradove kako bi što više menza bilo uključeno u ovakav način organizacije restorana
- digitalna mapa na kojoj će se odabirom drugog grada aplikacija preusmjeriti na tu lokaciju i prikazivati najbliže menze i njihove ponude u tom okruženju
- uvođenje dodatnog sistema međusobnog obavješćavanja studenata o veličini reda u pojedinoj menzi – studenti dolaskom u menzu javljaju u javni razgovor koliki je red (Nema reda, mali red, veliki red) u toj određenoj menzi te se u intervalima uzima sredina tih dojava
- dodatno obavješćavanje o količini dostupne hrane kako bi što točnije prenijeli informacije o ponudi u restoranu

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvatanja
F-001	Sustav omogućuje administratoru dodavanje novog jela ili brisanje postojećeg iz baze podataka.	Visok	Zahtjev dionika	Voditelj menze može dodavati i brisati jela kroz korisničko sučelje.
F-002	Sustav omogućuje administratoru kreiranje menzi te ispunjavanje njihovih osnovnih podataka.	Visok	Zahtjev dionika	Administrator može kreirati menzu i dodijeliti joj geografsku lokaciju, voditelja, sliku, radno vrijeme i slično.
F-003	Sustav omogućuje administratoru kreiranje korisničkih računa za voditelje menzi i radnike u menzi te povezivanje voditelja menze sa određenom menzom.	Visok	Zahtjev dionika	Administrator može kreirati obje vrste računa za zaposlenike u menzi.
F-004	Sustav omogućuje voditelju menze kreiranje dnevnog menija i određivanje radnog vremena.	Visok	Zahtjev dionika	Voditelj menze može kreirati dnevni meni iz jela koja se nalaze u bazi podataka te može postaviti radno vrijeme menze.
F-005	Sustav omogućuje radniku u menzi da promjeni stanje dostupnosti jela.	Visok	Zahtjev dionika	Radnik može mijenjati dostupnost određenog jela te također vrlo jednostavno može dodati novo dostupna jela.
F-006	Sustav omogućuje voditelju menze da šalje razne obavijesti radnicima i studentima.	Srednji	Zahtjev dionika	Voditelj menze može poslati interne obavijesti radnicima ili javne obavijesti studentima.
F-007	Sustav omogućuje radniku da vidi obavijesti koje mu je poslao voditelj menze.	Srednji	Povratne informacije radnika	Radnik može vidjeti obavijesti koje mu je poslao voditelj.

ID zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvaćanja
F-008	Sustav omogućuje vođitelju da vidi broj zainteresiranih studenata za određeno jelo.	Nisko	Zahtjev dionika	Voditelj menze može vidjeti mjeru zainteresiranosti studenata za određena jela.
F-009	Sustav omogućuje korisnicima prijavu putem Google računa.	Visok	Zahtjev dionika	Korisnik se može ulogirati u sustav putem Google računa umjesto e-mailom i lozinkom.
F-010	Sustav omogućuje korisnicima pregled trenutačno dostupnih jela u određenoj menzi.	Visok	Zahtjev dionika	Korisnik može vidjeti točan podatak o trenutačnoj dostupnosti jela u pojedinoj menzi.
F-011	Sustav obavještava korisnike o dostupnosti određenih jela na koje su se pretplatili.	Visok	Povratne informacije korisnika	Korisnici primaju push notifikacije o menzi gdje je jelo na koje su se pretplatili dostupno.
F-012	Sustav omogućuje korisnicima pretplatu na notifikacije dostupnosti određenih jela.	Visok	Zahtjev dionika	Korisnici mogu odabrati na koja se jela žele pretplatiti kako bi dobivali notifikacije o njihovoj dostupnosti.
F-013	Sustav omogućuje da se korisniku prikažu dostupne menze preko Google Mapsa.	Srednje	Postojeći sustav	Korisniku se otvara Google Maps kroz koji može vidjeti geografske lokacije menzi.
F-014	Sustav periodično nudi korisniku anketu o hrani koju bi volio vidjeti u menzi.	Nizak	Postojeći sustav	Korisniku je periodično predloženo sudjelovanje u kratkoj anketi u kojoj navode koja jela bi htjeli dodati/maknuti s menija.

ID zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvatanja
F-015	Sustav omogućuje korisnicima ocjenjivanje trenutačno dostupnih jela u određenoj menzi.	Nizak	Zahtjev dionika	Korisnik može ocijeniti trenutačno dostupno jelo, a tu ocjenu mogu vidjeti svi drugi korisnici.
F-016	Sustav omogućuje korisnicima informaciju o vremenu kada se promijenila dostupnost određenog jela.	Srednje	Zahtjev dionika	Korisnik može vidjeti vremensku oznaku trenutka kada je dostupno jelo postalo nedostupno ili obrnuto.

Nefunkcionalni zahtjevi

Zahtjevi za performance

ID zahtjeva	Opis	Prioritet
NF-1.1	Sustav treba imati vrijeme odziva manje od 3 sekunde.	Visok
NF-1.2	Sustav mora podržavati 1000 istovremenih korisnika i 10000 korisnika s aktivnom sjednicom unutar jednog sata.	Visok
NF-1.3	Kada radnik u menzi promijeni status dostupnosti jela, ta promjena mora biti vidljiva svim aktivnim studentima unutar 5 sekundi.	Srednji
NF-1.4	Sustav omogućuje automatsko horizontalno skaliranje.	Visok
NF-1.5	U slučaju naglog porasta prometa, sustav mora automatski alocirati nove resurse i stabilizirati performanse unutar definiranih granica.	Srednji
NF-1.6	Sustav mora imati optimizirano vrijeme učitavanja na mobilnoj mreži.	Srednje
NF-1.7	Korisničko sučelje mora biti u potpunosti funkcionalno.	Visok

Zahtjevi za sigurnost

ID zahtjeva	Opis	Prioritet
NF-2.1	Sustav mora striktno provoditi prava pristupa. "Radnik" ne smije moći pristupiti administrativnim funkcijama.	Visok

ID zahtjeva	Opis	Prioritet
NF-2.2	Podaci o pretplatama moraju biti enkriptirani i pohranjeni lokalno na mobilnom uređaju.	Srednji
NF-2.3	Svi API endpointi moraju biti dostupni samo autoriziranim korisnicima.	Visok

Zahtjevi za dostupnost

ID zahtjeva	Opis	Prioritet
NF-3.1	Sustav treba biti 99.5% dostupan od 9:00 do 21:00.	Visoki
NF-3.2	Aplikacija mora biti kompatibilna s Google Chrome, Mozilla Firefox i Apple Safari.	Visoki
NF-3.3	Aplikacija mora biti funkcionalna na operativnim sustavima iOS i Android.	Visoki
NF-3.4	Korisničko sučelje mora biti funkcionalno na svim širinama ekrana.	Visoki

Zahtjevi za održavanje

ID zahtjeva	Opis	Prioritet
NF-4.1	Mora postojati dokumentacija za administratore.	Srednji
NF-4.2	Svi API endpointi moraju biti dokumentirani.	Niski
NF-4.3	Ključni dijelovi koda moraju sadržavati komentare.	Srednji

Zahtjevi za korisničko iskustvo (UX)

ID zahtjeva	Opis	Prioritet
NF-5.1	Sustav treba imati intuitivno sučelje s jednostavnim pregledom otvorenih menzi i njihovom trenutačnom ponudom jela.	Srednji
NF-5.2	Sučelje mora biti responzivno.	Visoki
NF-5.3	Aplikacija mora omogućiti obavljanje osnovnih funkcija s minimalnim brojem koraka.	Srednji

Dionik 1: Korisnici

Korisnici su dionici koji koriste sustav. Podijeljeni su u tri kategorije:

- **Studenti:** Najbrojnija korisnička skupina. U interesu im je dobivanje pravovremenih informacija o ponudi u menzama

Aktivnosti: pregledavaju dostupnost jela, traže menze u blizini, pretplaćuju se na notifikacije za omiljena jela, ocjenjuju hranu

- **Radnici u menzi:** Odgovorni za ažuriranje podatka u sustavu. Njihov interes je jednostavno i brzo sučelje koje im ne ometa redovni posao

Aktivnosti: mijenjaju status dostupnosti pojedinih jela, primaju obavijesti od voditelja svoje menze

- **Voditelji menzi:** Odgovorni za upravljanje menzama. U interesu im je smanjenje bacnja hrane i povećanje zadovoljstva studenata

Aktivnosti: Postavljaju dnevne jelovnike i radno vrijeme, šalju obavijesti studentima i radnicima, prate ocjene pojedinih jela

Dionik 2: Administrator Sustava

Administrator ima najvišu razinu ovlasti i odgovoran je za funkcioniranje platforme.

Stvara, uređuje i briše profile menzi unutar sustava. Kreira korisničke račune za voditelje i radnike te ih hijerarhijski povezuje s odgovarajućim menzama.

Dionik 3: Razvojni tim

Razvojni tim je odgovoran za programiranje i održavanje aplikacije "Dish Detective".

Implementira korisničko sučelje i pozadinsku logiku. Dizajnira i upravlja bazom podataka. Integrira vanjske servise te je odgovoran za postavljanje i održavanje aplikacije na Vercel platformi.

Dionik 4: Naručitelj / Vlasnik Projekta

Naručitelj je organizacija u čijem je interesu rješavanje problema neažurnih jelovnika i time poboljšanje kvalitete studentske prehrane.

Definira temeljne ciljeve i opseg projekta. Osigurava potrebne resurse. Sudjeluje u ključnim odlukama tijekom razvoja i ima konačnu riječ pri odobravanju završenog proizvoda. Nadzire uspješnost projekta u odnosu na postavljene ciljeve.

A-1: Student (inicijator)

Student je registrirani korisnik koji pretražuje i prati ponudu menzi. Student je inicijator akcija poput ocjenjivanja, pretplate i interakcije s kartom.

Funkcionalnosti koje Student može obaviti:

- **Prikaz menzi, dostupnih jela i povratnih informacija.**
 - **Funkcionalni zahtjev: F-010** - Sustav omogućuje korisnicima pregled trenutno dostupnih jela u određenoj menzi.
 - **Funkcionalni zahtjev: F-013** - Sustav omogućuje da se korisniku prikažu dostupne menze preko Google Mapsa.
 - **Funkcionalni zahtjev: F-015** - Sustav omogućuje korisnicima ocjenjivanje trenutno dostupnih jela u određenoj menzi.
 - **Upravljanje obavijestima o jelima i anketiranje.**
 - **Funkcionalni zahtjev: F-012** - Sustav omogućuje korisnicima pretplatu na notifikacije dostupnosti određenih jela.
 - **Funkcionalni zahtjev: F-011** - Sustav obavještava korisnike o dostupnosti određenih jela na koje su se pretplatili.
 - **Funkcionalni zahtjev: F-014** - Sustav periodično nudi korisniku anketu o hrani koju bi volio vidjeti u menzi.
 - **Prijava u sustav.**
 - **Funkcionalni zahtjev: F-009** - Sustav omogućuje korisnicima prijavu putem Google računa.
-

A-2: Radnik u menzi (inicijator)

Radnik u menzi je ključni akter zadužen za ažuriranje stanja jelovnika u realnom vremenu.

Funkcionalnosti koje Radnik u menzi može obaviti:

- **Ažuriranje stanja dostupnih jela i automatsko obavještavanje studenata.**
 - **Funkcionalni zahtjev: F-005** - Sustav omogućuje radniku u menzi da promjeni stanje dostupnosti jela (te time obavijesti studente).
- **Pregled internih obavijesti.**

- **Funkcionalni zahtjev: F-007** - Sustav omogućuje radniku da vidi obavijesti koje mu je poslao voditelj menze.
-

A-3: Voditelj menze (inicijator i sudionik)

Voditelj menze odgovoran je za organizaciju ponude menze, postavljanje menija i komunikaciju sa zaposlenicima i studentima.

Funkcionalnosti koje Voditelj menze može obaviti:

- **Upravljanje podacima o jelima i kreiranje dnevnog menija.**
 - **Funkcionalni zahtjev: F-004** - Sustav omogućuje kreiranje dnevnog menija i određivanje radnog vremena.
 - **Komunikacija i nadzor.**
 - **Funkcionalni zahtjev: F-006** - Sustav omogućuje slanje raznih obavijesti radnicima i studentima.
 - **Funkcionalni zahtjev: F-008** - Sustav omogućuje uvid u broj zainteresiranih studenata za određeno jelo.
-

A-4: Administrator (inicijator)

Administrator upravlja sustavom, kreira menze, i uspostavlja hijerarhiju računa za zaposlenike.

Funkcionalnosti koje Administrator može obaviti:

- **Upravljanje menzama.**
 - **Funkcionalni zahtjev: F-001** - Sustav omogućuje dodavanje novog jela ili brisanje postojećeg iz baze podataka.
 - **Funkcionalni zahtjev: F-002** - Sustav omogućuje kreiranje menzi te ispunjavanje njihovih osnovnih podataka (lokacija, slika, voditelj).
- **Upravljanje korisničkim računima menze i hijerarhijom.**
 - **Funkcionalni zahtjev: F-003** - Sustav omogućuje kreiranje korisničkih računa za voditelje i radnike, te povezivanje voditelja s određenom menzom.

UC1 – Prijavi administratora

Element	Opis
Glavni sudionik	Administrator
Cilj	Omogućavanje administratoru prijavu te time pristup sustavu
Sudionici	Administrator, Sustav, Eksterni servis (Clerk)
Preduvjet	Administratorov račun se nalazi u sustavu
Opis osnovnog tijeka	1. Aktor na početnoj stranici odabire opciju login te potom opciju "zaposlenik" 2. Aplikacija preusmjerava aktora na Clerk stranicu za prijavu 3. Aktor unosi korisničko ime i lozinku u polja te odabire opciju za podnošenje prijave 4. Clerk provjerava odgovaraju li korisničko ime i lozinka postojećem administratorskom računu 5. Clerk omogućuje aktoru pristup sustavu
Opis mogućih odstupanja	• 4. a) Korisničko ime i lozinka ne odgovaraju niti jednom postojećem administratorskom računu ◦ Sustav obrađuje i ispisuje grešku, te se vraća na korak 2

UC2 – Prijavi voditelja i radnika menze

Element	Opis
Glavni sudionik	Voditelj ili radnik menze (ovisno o odabranoj opciji)
Cilj	Omogućuje voditelju ili radniku menze prijavu u sustav te pristup aplikaciji na razini određenoj vrstom računa
Sudionici	Voditelj ili radnik menze (ovisno o odabranoj opciji), Sustav, Eksterni servis (Clerk)
Preduvjet	Administrator je kreirao račun za voditelja/radnika
Opis osnovnog tijeka	1. Aktor na početnoj stranici odabire opciju login te potom opciju "zaposlenik" 2. Aplikacija preusmjerava aktora na Clerk stranicu za prijavu 3. Aktor unosi korisničko ime i lozinku u polja te odabire opciju za podnošenje prijave 4. Clerk provjerava odgovaraju li korisničko ime i lozinka postojećem računu unutar baze voditelja menzi ili radnika u menzi 5. Clerk omogućuje aktoru pristup sustavu
Opis mogućih odstupanja	• 4. a) Korisničko ime i lozinka ne odgovaraju niti jednom postojećem računu voditelja menze ili radnika u menzi(ovisno o odabranoj opciji) ◦ Sustav obrađuje i ispisuje grešku, te se vraća na korak 2

UC3 – Prijavi studenta

Element	Opis
Glavni sudionik	Student
Cilj	Omogućuje studentu u sustav te pristup aplikaciji
Sudionici	Student, Sustav, Eksterni servis (Clerk), Eksterni servis (Google OAuth 2.0)
Preduvjet	Student ima Google account
Opis osnovnog tijeka	1. Aktor na početnoj stranici odabire opciju login te potom opciju "student" 2. Aplikacija preusmjerava korisnika na Clerk stranicu za autentifikaciju 3. Clerk zatim preusmjerava korisnika na Google stranicu za autentifikaciju 4. Aktor unosi potrebne podatke te odabire opciju za podnošenje prijave 5. Aktor odobrava aplikaciji pristup podacima s Google računa 6. Aplikacija provjerava postoji li student u bazi. Ako ne postoji, vrši se registracija, ako postoji, vrši se prijava 7. Ako je registracija, aplikacija pohranjuje korisničke podatke i završava registraciju 8. Ako je prijava, aplikacija autentificira aktora i omogućuje pristup aplikaciji
Opis mogućih odstupanja	• 4. a) Aktor ne odobrava pristup podacima sa Google računa ◦ Google obrađuje pogrešku i ispisuje poruku da je korisnik odbio pristup, te se vraća na korak 1 • 4. b) Tehnička greška tijekom registracije/prijave ◦ Google ili Clerk obrađuje pogrešku i ispisuje poruku da je došlo do tehničke greške, te se vraća na korak 1

UC4 – Kreacija menze od strane admina

Element	Opis
Glavni sudionik	Administrator
Cilj	Omogućiti administratoru dodavanje nove menze u sustav.
Sudionici	Administrator, Sustav
Preduvjet	Administrator je prijavljen u sustav.
Opis osnovnog tijeka	1. Administrator u sučelju odabire opciju za upravljanje menzama. 2. Administrator odabire opciju "Dodaj novu menzu". 3. Aplikacija prikazuje formu za unos podataka o novoj menzi. 4. Administrator unosi podatke (naziv, adresa, kontakt, radno vrijeme). 5. Administrator potvrđuje unos. 6. Sustav sprema novu menzu u bazu podataka i prikazuje potvrdu.

Element	Opis
Opis mogućih odstupanja	4. a) Administrator unese neispravne ili nepotpune podatke. - Sustav ispisuje poruku o grešci specificirajući koja polja nisu ispravna. Tijek se vraća na korak 3.

UC5 - Dodjela voditelja menzi od strane administratora

Element	Opis
Glavni sudionik	Administrator
Cilj	Omogućiti administratoru da postojećeg voditelja postavi kao voditelja određene menze.
Sudionici	Administrator, Sustav
Preduvjet	Administrator je prijavljen. Postoji barem jedna menza bez voditelja i postoji taj voditelj.
Opis osnovnog tijeka	- Administrator odabire opciju za upravljanje menzama. - Administrator odabire menzu kojoj želi dodijeliti voditelja. - Administrator odabire opciju "Dodijeli voditelja". - Sustav prikazuje listu postojećih korisnika koji mogu biti voditelji. - Administrator odabire željenog korisnika s liste. - Sustav dodjeljuje odabranom korisniku ulogu voditelja menze i prikazuje potvrdu.
Opis mogućih odstupanja	2. a) Odabrana menza već ima dodijeljenog voditelja. - Sustav ispisuje obavijest i nudi opciju promjene postojećeg voditelja.

UC6 - Dodavanje i brisanje jela iz baze podataka (admin)

Element	Opis
Glavni sudionik	Administrator
Cilj	Upravljanje centralnom bazom svih jela dostupnih u sustavu.
Sudionici	Administrator, Sustav
Preduvjet	Administrator je prijavljen u sustav.

Element	Opis
Opis osnovnog tijeka (Dodavanje)	1. Administrator odabire opciju za upravljanje jelima. 2. Administrator odabire "Dodaj novo jelo". 3. Aplikacija prikazuje formu za unos podataka o jelu (naziv, opis, alergeni, nutritivne vrijednosti). 4. Administrator unosi podatke i potvrđuje unos. 5. Sustav sprema novo jelo u bazu podataka.
Opis osnovnog tijeka (Brisanje)	1. Administrator odabire opciju za upravljanje jelima. 2. Sustav prikazuje listu svih jela. 3. Administrator odabire jelo koje želi obrisati. 4. Administrator odabire opciju "Obriši". 5. Sustav traži potvrdu brisanja. 6. Administrator potvrđuje brisanje. 7. Sustav briše jelo iz baze podataka.
Opis mogućih odstupanja	• Dodavanje 4. a) Jelo s istim nazivom već postoji. ◦ Sustav ispisuje poruku o grešci. Tijek se vraća na korak 3 (Dodavanje). • Brisanje 3. a) Jelo koje se briše je dio aktivnog menija. ◦ Sustav ispisuje upozorenje i ne dopušta brisanje dok se jelo ne ukloni iz svih menija.

UC7 - Kreacija dnevnog menija i određivanje radnog vremena

Element	Opis
Glavni sudionik	Voditelj menze
Cilj	Omogućiti voditelju menze da kreira dnevni meni za svoju menzu i postavi specifično radno vrijeme.
Sudionici	Voditelj menze, Sustav
Preduvjet	Voditelj menze je prijavljen u sustav.
Opis osnovnog tijeka	1. Voditelj menze odabire opciju "Upravljanje menijem". 2. Odabire datum za koji želi kreirati ili urediti meni. 3. Sustav prikazuje listu svih dostupnih jela iz centralne baze. 4. Voditelj odabire jela koja će biti na meniju za taj dan. 5. Za svako odabrano jelo postavlja početnu dostupnost (npr. dostupno, nedostupno). 6. Voditelj postavlja radno vrijeme menze za odabrani datum. 7. Voditelj sprema dnevni meni. 8. Sustav objavljuje meni za odabrani datum.
Opis mogućih odstupanja	• 2. a) Za odabrani datum već postoji kreiran meni. ◦ Sustav učitava postojeći meni i omogućuje njegovo uređivanje.

UC8 – Mijenjanje dostupnosti jela (radnik)

Element	Opis
Glavni sudionik	Radnik menze
Cilj	Omogućiti radniku brzo i jednostavno ažuriranje statusa dostupnosti pojedinog jela na dnevnom meniju.
Sudionici	Radnik menze, Sustav
Preduvjet	Radnik je prijavljen u sustav. Voditelj je kreirao dnevni meni.
Opis osnovnog tijeka	1. Radnik se prijavljuje i odabire svoju menzu. 2. Aplikacija prikazuje današnji meni s trenutnim statusom dostupnosti za svako jelo. 3. Radnik uočava da je neko jelo potrošeno ili ponovno dostupno. 4. Radnik pronalazi to jelo na listi i mijenja njegov status (npr. iz "Dostupno" u "Nedostupno"). 5. Sustav trenutno ažurira status jela, što postaje vidljivo svim studentima.
Opis mogućih odstupanja	• 2. a) Za današnji dan nije kreiran meni. ◦ Sustav ispisuje poruku da meni nije postavljen.

UC9 – Slanje obavijesti radnicima i studentima

Element	Opis
Glavni sudionik	Voditelj menze
Cilj	Omogućiti voditelju slanje važnih obavijesti svim radnicima svoje menze i/ili svim studentima koji prate menzu.
Sudionici	Voditelj menze, Studenti, Radnici, Sustav
Preduvjet	Voditelj menze je prijavljen u sustav.
Opis osnovnog tijeka	1. Voditelj menze odabire opciju "Pošalji obavijest". 2. Odabire ciljanu publiku (npr. "Radnici", "Studenti", "Svi"). 3. Unosi naslov i tekst obavijesti. 4. Odabire opciju "Pošalji". 5. Sustav šalje obavijest odabranoj grupi korisnika putem notifikacija unutar aplikacije.
Opis mogućih odstupanja	• 3. a) Tekst obavijesti je prazan. ◦ Sustav ispisuje grešku i ne dopušta slanje. Tijek se vraća na korak 3.

UC10 – Pregled obavijesti (studenti i radnici)

Element	Opis
Glavni sudionik	Student ili Radnik menze
Cilj	Omogućiti studentima i radnicima pregled primljenih obavijesti.
Sudionici	Student, Radnik menze, Sustav
Preduvjet	Korisnik je prijavljen u sustav.
Opis osnovnog tijeka	1. Korisnik u aplikaciji odabire ikonu ili opciju za obavijesti. 2. Sustav prikazuje listu svih primljenih obavijesti, kronološki poredanih. 3. Korisnik može odabrati pojedinu obavijest kako bi je pročitao u cijelosti. 4. Sustav označava pročitane obavijesti.
Opis mogućih odstupanja	• 1. a) Nema novih obavijesti. ◦ Sustav prikazuje poruku "Nemate novih obavijesti".

UC11 - Pregled ljudi zainteresiranih za neko jelo (voditelj)

Element	Opis
Glavni sudionik	Voditelj menze
Cilj	Omogućiti voditelju menze uvid u to koliko je studenata pretplaćeno na obavijesti za određena jela.
Sudionici	Voditelj menze, Sustav
Preduvjet	Voditelj menze je prijavljen u sustav.
Opis osnovnog tijeka	1. Voditelj menze odabire opciju "Statistika". 2. Sustav prikazuje listu svih jela iz baze podataka. 3. Pored svakog jela, sustav prikazuje broj studenata koji su se pretplatili na obavijesti za to jelo. 4. Voditelj može sortirati listu po popularnosti kako bi lakše planirao buduće menije.
Opis mogućih odstupanja	• Nema značajnih odstupanja.

UC12 - Pregled trenutno dostupnih jela u određenoj menzi

Element	Opis
Glavni sudionik	Student

Element	Opis
Cilj	Omogućiti studentu da u stvarnom vremenu vidi koja su jela dostupna u odabranoj menzi.
Sudionici	Student, Sustav
Preduvjet	Student je prijavljen u sustav.
Opis osnovnog tijeka	1. Student otvara aplikaciju i odabire menzu koju želi provjeriti. 2. Aplikacija prikazuje dnevni meni za odabranu menzu. 3. Pored svakog jela stoji oznaka o dostupnosti (npr. "Dostupno", "Nedostupno"). 4. Prikaz se automatski ažurira kako radnici mijenjaju status dostupnosti.
Opis mogućih odstupanja	• 1. a) Za odabranu menzu danas nije postavljen meni. ◦ Aplikacija prikazuje poruku "Meni za danas još nije objavljen". • 1. b) Trenutno vrijeme ne upada u radno vrijeme odabrane menze. ◦ Aplikacija prikazuje poruku "Menza ne radi".

UC13 – Pretplata na jelo (student)

Element	Opis
Glavni sudionik	Student
Cilj	Omogućiti studentu da se pretplati na obavijesti za omiljeno jelo kako bi bio informiran kada postane dostupno.
Sudionici	Student, Sustav
Preduvjet	Student je prijavljen u sustav.
Opis osnovnog tijeka	1. Student pregledava listu svih mogućih jela u aplikaciji. 2. Pored jela koje ga zanima, student odabire opciju "Pretplati se". 3. Sustav sprema informaciju o pretplati za tog studenta i to jelo. 4. Student može na isti način otkazati pretplatu.
Opis mogućih odstupanja	• 2. a) Student je već pretplaćen na odabrano jelo. ◦ Sustav nudi opciju za otkazivanje pretplate.

UC14 – Notifikacija o jelu (sustav)

Element	Opis
Glavni sudionik	Sustav
Cilj	Automatski obavijestiti sve pretplaćene studente kada jelo na koje su pretplaćeni postane dostupno u nekoj od menzi.

Element	Opis
Sudionici	Sustav, Student, Eksterni servis (Firebase Cloud Messaging)
Preduvjet	Barem jedan student je pretplaćen na barem jedno jelo.
Opis osnovnog tijeka	1. Voditelj menze objavi dnevni meni koji sadrži jelo "X". 2. Sustav provjerava listu svih studenata koji su pretplaćeni na jelo "X". 3. Sustav automatski šalje push notifikaciju svim tim studentima s porukom (npr. "Sarma je danas dostupna u Menzi Z!").
Opis mogućih odstupanja	• Nema značajnih odstupanja u ovom automatiziranom procesu.

UC15 - Pinganje menzi preko Google Mapsa

Element	Opis
Glavni sudionik	Student
Cilj	Omogućiti studentu da lako pronađe lokaciju menze i dobije upute za dolazak.
Sudionici	Student, Sustav, Eksterni servis (Google Maps)
Preduvjet	Student ima pristup internetu i lokacijskim uslugama na svom uređaju.
Opis osnovnog tijeka	1. Student unutar aplikacije pregledava detalje o određenoj menzi. 2. Student odabire opciju "Prikaži na karti" ili "Navigacija". 3. Aplikacija prosljeđuje koordinate menze vanjskoj aplikaciji za mape (npr. Google Maps). 4. Vanjska aplikacija se otvara i prikazuje lokaciju menze te nudi opcije za navigaciju od trenutne lokacije studenta.
Opis mogućih odstupanja	• 3. a) Student nema instaliranu aplikaciju za mape. ◦ Sustav nudi otvaranje lokacije u web pregledniku.

UC16 - Sustav periodično nudi anketu

Element	Opis
Glavni sudionik	Sustav
Cilj	Prikupljanje povratnih informacija od studenata o zadovoljstvu hranom i uslugom.
Sudionici	Sustav, Student
Preduvjet	Administrator je kreirao i aktivirao anketu u sustavu.

Element	Opis
Opis osnovnog tijeka	1. Sustav, prema unaprijed definiranim pravilima (npr. jednom mjesečno), nasumično odabire aktivne studente. 2. Sustav tim studentima šalje notifikaciju s pozivom za ispunjavanje ankete. 3. Notifikacija sadrži link koji vodi direktno na anketu unutar aplikacije.
Opis mogućih odstupanja	• Nema značajnih odstupanja u ovom automatiziranom procesu.

UC17 - Ispunjavanje ponuđene ankete o hrani

Element	Opis
Glavni sudionik	Student
Cilj	Omogućiti studentu da izrazi svoje mišljenje i prijedloge kroz anketu.
Sudionici	Student, Sustav
Preduvjet	Student je primio poziv za ispunjavanje ankete.
Opis osnovnog tijeka	1. Student klikne na notifikaciju ili link za anketu. 2. Aplikacija prikazuje anketu s pitanjima (npr. ocjena kvalitete, prijedlozi za nova jela). 3. Student odgovara na pitanja. 4. Student podnosi ispunjenu anketu. 5. Sustav sprema odgovore anonimno u bazu podataka za kasniju analizu.
Opis mogućih odstupanja	• 1. a) Student ignorira ili odbije ispuniti anketu. ◦ Sustav bilježi da je anketa ponuđena i ne nudi je ponovno određeni period.

UC18 - Ocjenjivanje jela u menzi

Element	Opis
Glavni sudionik	Student
Cilj	Omogućiti studentu da ocijeni pojedino jelo nakon konzumacije.
Sudionici	Student, Sustav
Preduvjet	Student je prijavljen. Jelo je na dnevnom meniju.

Element	Opis
Opis osnovnog tijeka	1. Student pregledava meni unutar aplikacije. 2. Pored svakog jela, student vidi trenutnu prosječnu ocjenu i opciju za ocjenjivanje (npr. zvjezdice). 3. Student odabire jelo koje želi ocijeniti. 4. Student dodjeljuje ocjenu (npr. od 1 do 5 zvjezdica). 5. Sustav bilježi ocjenu i ažurira prosječnu ocjenu jela.
Opis mogućih odstupanja	• 4. a) Student je već ocijenio to jelo u određenom vremenskom periodu (npr. danas). ◦ Sustav prikazuje njegovu postojeću ocjenu i omogućuje mu da je promijeni, ali ne i da doda novu.

UC19 - Prikaz vremena promjene dostupnosti jela

Element	Opis
Glavni sudionik	Student
Cilj	Omogućiti studentu informaciju o točnom vremenu kada je status dostupnosti nekog jela zadnji put promijenjen.
Sudionici	Student, Sustav
Preduvjet	Student je prijavljen u sustav. Radnik menze je promijenio dostupnost barem jednog jela na današnjem meniju.
Opis osnovnog tijeka	1. Student unutar aplikacije odabere menzu i pregledava današnji meni. 2. Radnik menze promijeni status jela iz "Dostupno" u "Nedostupno" (ili obrnuto). 3. Sustav zabilježi točno vrijeme promjene statusa. 4. Prilikom pregleda menija, student uz status jela (npr. "Nedostupno") vidi i vremensku oznaku kada je ta promjena nastupila (npr. "od 14:20").
Opis mogućih odstupanja	• 4. a) Status jela se nije mijenjao od početka dana (od objave menija). ◦ Sustav prikazuje samo status "Dostupno" bez dodatne vremenske oznake.

UC20 - Dodavanje i brisanje računa za radnike iz baze podataka(admin)

Element	Opis
Glavni sudionik	Administrator
Cilj	Upravljanje korisničkim računima radnika menze i određivanje voditelja.

Element	Opis
Sudionici	Administrator, Sustav
Preduvjet	Administrator je prijavljen u sustav. Radnici čiji se račun briše prijašnje postoji u bazi podataka. Radnici koji se dodaju u bazu još ne postoje u njoj.
Opis osnovnog tijeka (Dodavanje)	- Administrator odabire opciju za upravljanje računima radnika menze. - Administrator odabire gumb s oznakom dodavanja (+). - Sustav prikazuje formu za unos podataka o radniku (ime, prezime, korisničko ime, lozinka, menza kojoj pripada, uloga). - Administrator unosi potrebne podatke. - Administrator potvrđuje unos. - Sustav sprema novog radnika u bazu podataka. - Sustav prikazuje poruku o uspješnom dodavanju računa.
Opis osnovnog tijeka (Brisanje)	- Administrator odabire opciju za upravljanje računima radnika menze. - Sustav prikazuje listu svih radnika menze. - Administrator odabire radnika kojeg želi obrisati. - Administrator odabire gumb s oznakom brisanja (Trashcan). - Sustav traži potvrdu brisanja. - Administrator potvrđuje brisanje. - Sustav briše račun radnika iz baze podataka.
Opis mogućih odstupanja	4. a) Uneseno korisničko ime već postoji u sustavu. - Sustav ispisuje poruku o grešci i vraća se na korak 3 (dodavanje). 5. a) Administrator pokušava postaviti voditelja u menzi koja već ima voditelja. - Sustav ispisuje poruku o grešci.

3. Specifikacija zahtjeva sustava

1. Visokorazinski dijagram obrazaca uporabe cijelog sustava

- Dijagram sustava 1
- Dijagram sustava 2

2. Dijagram obrazaca uporabe za ključne značajke

- Prijava korisnika

3. Dijagram obrazaca uporabe za korisničke uloge

- ▶ Funkcionalnost administratora
- ▶ Funkcionalnost radnika i voditelja menze
- ▶ Funkcionalnost studenta

Dijagram obrazaca uporabe za osnovne poslovne procese

- ▶ Osnovni poslovni procesi menze

Dijagram obrazaca uporabe za kritične sustave i integracije

- ▶ UC15 - Pinganje menzi preko Google Mapsa

Sekvencijski dijagrami

- ▶ Prijava i registracija studenta
- ▶ Slanje obavijesti o dostupnosti hrane studentima
- ▶ Slanje generalne obavijesti
- ▶ Dodavanje i brisanje jeela

Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

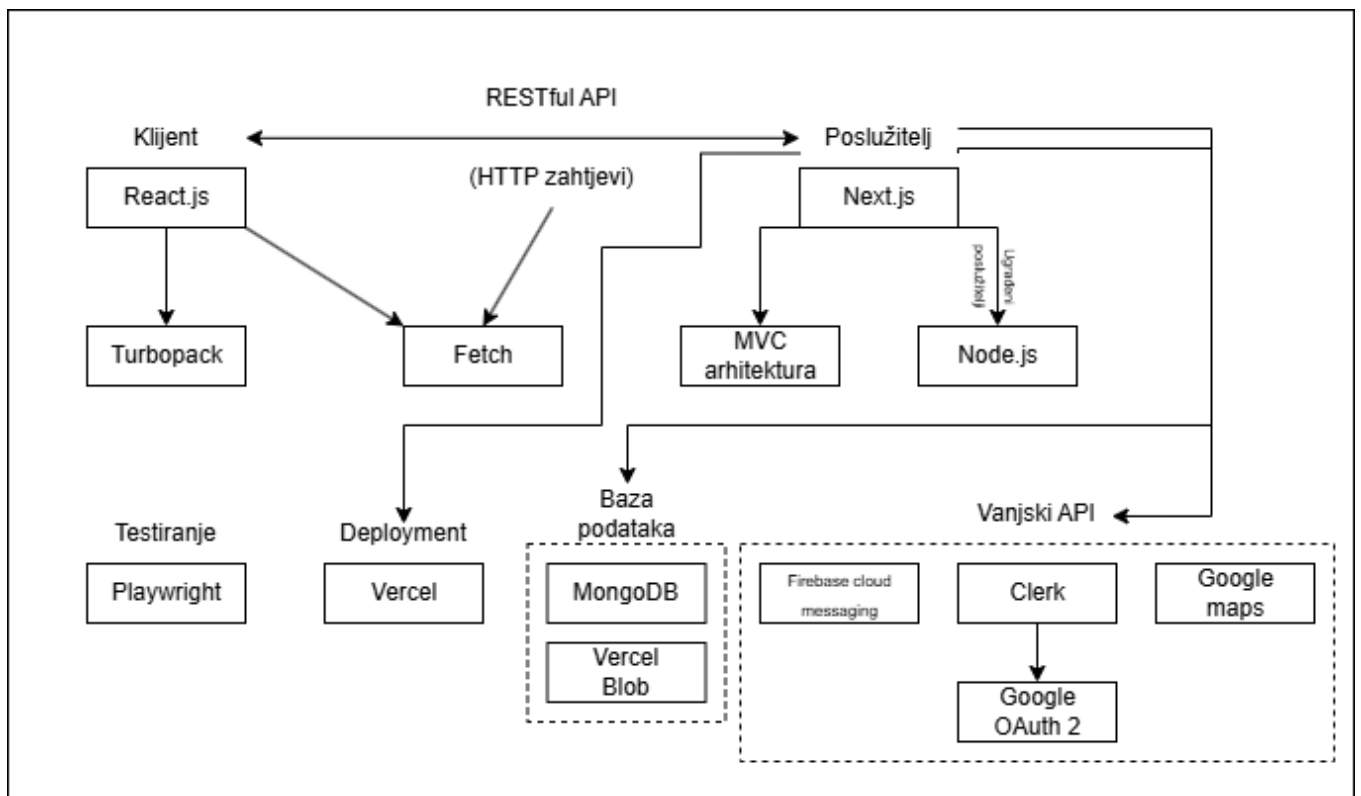
Redni broj	Naziv obrasca uporabe	Funkcionalni zahtjevi
UC1	Prijavi administratora	
UC2	Prijavi voditelja i radnika menze	
UC3	Prijavi studenta	F-009
UC4	Kreacija menze od strane admina	F-002
UC5	Dodjela voditelja menzi (admin)	F-003

Redni broj	Naziv obrasca uporabe	Funkcionalni zahtjevi
UC6	Dodavanje i brisanje jela iz baze podataka (admin)	F-001
UC7	Kreacija dnevnog menija i određivanje radnog vremena	F-004
UC8	Mijenjanje dostupnosti jela (radnik)	F-005
UC9	Slanje obavijesti radnicima i studentima	F-006
UC10	Pregled obavijesti (studenti i radnici)	F-007
UC11	Pregled ljudi zainteresiranih za neko jelo (voditelj)	F-008
UC12	Pregled trenutno dostupnih jela u određenoj menzi	F-010
UC13	Pretplata na jelo (student)	F-011, F-012
UC14	Notifikacija o jelu (sustav)	F-011, F-012
UC15	Pinganje menzi preko Google Mapsa	F-013
UC16	Sustav periodično nudi anketu	F-014
UC17	Ispunjavanje ponuđene ankete o hrani	F-014
UC18	Ocjenjivanje jela u menzi	F-015
UC19	Prikaz vremena promjene dostupnosti jela	F-016
UC20	Dodavanje i brisanje računa za radnike iz baze podataka(admin)	F-003

Arhitektura sustava

Opis arhitekture

- Stil arhitekture:** Na najvišoj razini, arhitektura aplikacije prati **klijent-poslužitelj** model. Odabrali smo ovaj stil jer omogućuje jasno razdvajanje korisničkog sučelja od poslovne logike i baze podataka. Poslužitelj centralno upravlja podacima, što olakšava održavanje, osigurava konzistentnost podataka i povećava sigurnost. Klijentska aplikacija je time rasterećena i fokusirana na pružanje kvalitetnog korisničkog iskustva.



Slika 4.1: Vizualizacija klijent-poslužitelj arhitekture

- **Podsustavi:**

1. **Klijent (Frontend):** Korisničko sučelje razvijeno korištenjem **React.js** i **Next.js** frameworka. Odgovorno je za prikaz podataka korisnicima (jelovnici, lokacije menzi) i omogućavanje interakcije (prijava, ostavljanje recenzija).
2. **Poslužitelj (Backend):** Implementiran unutar **Next.js** frameworka, koji omogućuje stvaranje API endpointa. Poslužitelj obrađuje sve korisničke zahtjeve, upravlja poslovnom logikom (npr. ažuriranje jelovnika, slanje notifikacija) i komunicira s bazom podataka.
3. **Baza podataka:** Koristi se **MongoDB**, nerelacijska (NoSQL) baza podataka, za pohranu svih podataka aplikacije.
4. **Dodatni servisi:**
 - **Clerk:** Za autentifikaciju i upravljanje korisnicima.
 - **Vercel Blob:** Za pohranu i posluživanje slika (npr. slike jela i menzi).
 - **Firebase Cloud Messaging:** Za slanje notifikacija korisnicima o promjenama jelovnika ili obavijestima menze.
 - **Google Maps API:** Za prikaz lokacija menzi.

- **Preslikavanje na radnu platformu:** Za deployment aplikacije odabran je **Vercel**, cloud platforma optimizirana za Next.js aplikacije. Vercel omogućuje automatski proces izgradnje (build) i distribucije (deploy) aplikacije direktno s GitHuba, što osigurava kontinuiranu isporuku (CI/CD).
- **Spreništa podataka:** Podaci su pohranjeni u **MongoDB** bazi. Za slike i druge statičke datoteke koristi se **Vercel Blob**. Odabran je nerelacijski model zbog fleksibilnosti sheme i optimizacije za specifične upite aplikacije.

- **Mrežni protokoli:** Klijent komunicira s poslužiteljem putem **HTTPS** protokola, koristeći RESTful API principe za slanje zahtjeva i primanje odgovora.
- **Globalni upravljački tok:**
 1. Korisnik otvara web aplikaciju u pregledniku.
 2. Klijentski dio aplikacije (React/Next.js) šalje HTTPS zahtjev poslužiteljskom dijelu (Next.js API) za dohvat podataka (npr. jelovnika za odabranu menzu).
 3. Poslužitelj obrađuje zahtjev, poziva odgovarajuću logiku i šalje upit MongoDB bazi podataka.
 4. Baza podataka vraća tražene podatke poslužitelju.
 5. Poslužitelj formatira podatke u JSON format i šalje ih kao odgovor klijentu.
 6. Klijent prikazuje primljene podatke korisniku (uključujući slike čiji se URL-ovi nalaze u podacima).
- **Sklopovsko-programski zahtjevi:** Za korištenje aplikacije, klijentu je potreban samo moderan web preglednik. Poslužiteljska strana se izvršava na Vercel platformi, čime su apstrahirani direktni hardverski zahtjevi.

Obrazloženje odabira arhitekture

- **Izbor arhitekture temeljen na principima oblikovanja:** Korištenje Next.js frameworka za frontend i backend unutar klijent-poslužitelj modela odabrano je zbog:
 1. **Jednostavnost razvoja:** Next.js omogućuje razvoj korisničkog sučelja i API-ja unutar istog projekta, što ubrzava razvoj i smanjuje kompleksnost.
 2. **Performanse:** Next.js nudi optimizacije poput Server-Side Rendering (SSR) i Static Site Generation (SSG), što rezultira bržim učitavanjem stranica.
 3. **Skalabilnost:** Vercel platforma omogućuje jednostavno skaliranje resursa prema potrebi, a MongoDB je poznat po svojoj horizontalnoj skalabilnosti.
 4. **Fleksibilnost:** Odvajanje klijenta i poslužitelja omogućuje neovisan razvoj i potencijalnu zamjenu pojedinih dijelova sustava u budućnosti (npr. izrada mobilne aplikacije koja bi koristila isti backend).
- **Razmatrane alternative:**
 1. **Tradicionalna višeslojna arhitektura:** Razmatrana je opcija s potpuno odvojenim backendom (npr. u Node.js/Express) i frontendom (React). Odustalo se od toga jer Next.js nudi integrirano rješenje koje je za potrebe ovog projekta jednostavnije i brže za razvoj.
 2. **Relacijska baza podataka (npr. PostgreSQL):** Relacijske baze su također bile opcija, ali MongoDB je odabran zbog veće fleksibilnosti i mogućnosti optimizacije sheme za specifične, česte upite (poput dohvata pretplatnika).

Organizacija sustava na visokoj razini

- **Klijent-poslužitelj:** Sustav naše aplikacije Dish Detective dizajniran je i implementiran korištenjem klijent-poslužitelj arhitekture koja se modelira na način da klijent pristupa poslužitelju tražeći uslugu, a poslužitelj ju dostavlja. Funkcionalnosti su jasno raspoređene između klijenta i poslužitelja čineći ovaj sistem učinkovitim u smanjenju opterećenja, povećavanju modularnosti, responzivnosti i mogućnosti paralelnog razvoja, što je ključno za pravovremeno informiranje studenata o stanju u menzama. Komponente su razdvojene te se mogu razvijati neovisno jedna o drugoj što omogućuje bolju prilagodbu novim zahtjevima i promjenama u razvoju, a takve su sposobnosti ključne u tehnologiji za rast bez gubitka kvalitete.
- **Baza podataka:** Koristi se MongoDB, dokumentno-orijentirana, NoSQL baza podataka. Ovakav odabir omogućuje fleksibilnost u pohrani različitih tipova podataka (npr. GeoJSON za lokacije menzi i nizovi za pretplate). Uloga baze je osigurati stalnost podataka o korisnicima, menzama, jelima i pretplatama, uz visoku učinkovitost kod čestih operacija čitanja (npr. prilikom pregleda ponude restorana).
- **Datotečni sustav:** Za pohranu binarnih podataka (slike jela i menzi) sustav ne koristi lokalni disk, već Vercel Blob servis. To omogućuje brzu isporuku statičkog sadržaja putem mreže, čime se smanjuje opterećenje glavnog poslužitelja i ubrzava učitavanje na mobilnim mrežama.
- **Grafičko sučelje:** Implementirano je kao responzivna web aplikacija pristupačna putem preglednika. Frontend je oblikovan korištenjem Reacta koji se oslanja na HTML, CSS i JavaScript za dinamički prikaz podataka i izgled stranice. Povezano je s backendom putem RESTful API-ja koristeći HTTP zahtjev.

Organizacija aplikacije

- **Frontend sloj:** Izgrađen je od nezavisnih React komponenti koje upravljaju vlastitim stanjem i prikazom. Umjesto da se cijela stranica renderira odjednom, svaka komponenta upravlja sobom te učitava sama za sebe. Ovakav pristup omogućuje veću neovisnost i dinamičko osvježavanje elemenata.
- **Backend sloj:** Implementiran je kroz Next.js Server Actione koji služe kao funkcionalni posrednici pri dohvat podataka iz baze MongoDB i komunikaciji s vanjskim servisima.
- **MVC arhitektura:** U širem smislu mogu se prepoznati dijelovi MVC pristupa, ali implementacija Dish Detective aplikacije odmiče od klasične MVC arhitekture. Možemo prepoznati
 - **Model:** podaci su definirani Mongoose shemama u bazi podataka, ali njihovo korištenje unutar aplikacije nije centralizirano u jednom kontroleru, već se distribuira tamo gdje je komponenti potrebno.
 - **View & Controller (Komponente):** U Reactu, granica između pogleda (View) i logike (Controller) nestaje unutar komponente. Komponenta u sebi sadrži i logiku za upravljanje podacima i kod za njihovo prikazivanje, što pruža veliku fleksibilnost u razvoju kompleksnih sučelja.

- **Modularnost:** Ovakav pristup omogućuje odvajanje poslovne logike od korisničkog sučelja na razini komponente, a ne na razini cijele aplikacije, što olakšava održavanje sustava i dodavanje novih funkcionalnosti, poput novih tipova anketa ili ocjenjivanja.

Baza podataka

Poslužitelj Clerk služi i za autentifikaciju ali i za spremanje metapodataka o korisnicima, u Clerku se spremaju naredni podatci o korisnicima:

Atribut	Tip podatka	Opis
<code>role</code>	<code>String</code>	Uloga korisnika unutar aplikacije (<code>'student'</code> , <code>'worker'</code> , <code>'manager'</code> , <code>'admin'</code>)
<code>restaurantId</code>	<code>String</code>	(Samo ako je role worker ili manager) Id restorana kojem radnik ili voditelj pripadaju

Baza podataka temelji se na nerelacijskom (NoSQL) modelu, implementiranom u **MongoDB**. Shema je dizajnirana tako da optimizira najčešće operacije čitanja, kao što je dohvaćanje pretplatnika za određeno jelo.

Baza podataka uključuje sljedeće kolekcije:

- `restaurants`
- `dishes`
- `menus`
- `reviews`
- `dishSubscriptions`
- `announcements`
- `polls`
- `answers`
- `readReceipts`
- `allergens`

Opis kolekcija

`restaurants`

restaurants: Kolekcija koja sadrži podatke o studentskim menzama.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator menze
<code>name</code>	<code>String</code>	Naziv menze
<code>address</code>	<code>String</code>	Adresa menze
<code>location</code>	<code>Object</code>	GeoJSON objekt: <code>{ type: 'Point', coordinates: [longitude, latitude] }</code>
<code>workingHours</code>	<code>Array</code>	Niz objekata s radnim vremenom: <code>[{ day: Number, shifts: [{ start: String, end: String }] }]</code>
<code>imageUrl</code>	<code>String</code>	URL slike menze
<code>availableDishes</code>	<code>Array</code>	Niz <code>ObjectId</code> referenci na <code>dishes</code> koje ovaj restoran nudi

dishes

dishes: Kolekcija koja sadrži podatke o pojedinačnim jelima.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator jela
<code>name</code>	<code>String</code>	Naziv jela
<code>description</code>	<code>String</code>	Opis jela
<code>imageUrl</code>	<code>String</code>	URL slike jela
<code>allergens</code>	<code>Array</code>	Niz <code>ObjectId</code> referenci na <code>allergens</code> kolekciju

menus

menus: Kolekcija koja definira dnevnu ponudu menze.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator jelovnika
<code>restaurantId</code>	<code>ObjectId</code>	Referenca na <code>restaurants</code> kolekciju

Atribut	Tip podatka	Opis
<code>date</code>	<code>Date</code>	Datum za koji jelovnik vrijedi
<code>items</code>	<code>Array</code>	Niz stavki na meniju: <code>[{ dishId: ObjectId, available: Boolean, lastServed: Date }]</code>
<code>lastUpdatedBy</code>	<code>String</code>	ID korisnika koji je zadnji ažurirao meni

reviews (DishRating)

reviews: Kolekcija za pohranu ocjena jela.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator ocjene
<code>dishId</code>	<code>String</code>	Referenca na <code>dishes</code> kolekciju
<code>restaurantId</code>	<code>String</code>	Referenca na <code>restaurants</code> kolekciju
<code>userId</code>	<code>String</code>	ID korisnika koji je dao ocjenu
<code>rating</code>	<code>Number</code>	Numerička ocjena

dishSubscriptions (Subscriptions)

dishSubscriptions: Kolekcija za pretplate studenata na obavijesti o jelima.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator pretplate
<code>userId</code>	<code>String</code>	Clerk ID korisnika
<code>dishId</code>	<code>ObjectId</code>	Referenca na <code>dishes</code> kolekciju
<code>restaurantId</code>	<code>ObjectId</code>	Referenca na <code>restaurants</code> kolekciju
<code>createdAt</code>	<code>Date</code>	Vrijeme kreiranja pretplate

announcements (Notifications)

announcements: Kolekcija za pohranu obavijesti i notifikacija.

Atribut	Tip podatka	Opis
<code>_id (PK)</code>	<code>ObjectId</code>	Jedinstveni identifikator obavijesti
<code>title</code>	<code>String</code>	Naslov obavijesti (default: <code>'Obavijest'</code>)
<code>description</code>	<code>String</code>	Tekst obavijesti
<code>type</code>	<code>String</code>	Tip obavijesti: <code>'worker'</code> ili <code>'student'</code>
<code>postedBy</code>	<code>String</code>	ID korisnika koji je kreirao obavijest
<code>read</code>	<code>Boolean</code>	Status pročitivosti (default: <code>false</code>)
<code>targetUserId</code>	<code>String</code>	(Opcionalno) ID specifičnog studenta kojem je namijenjena
<code>restaurantId</code>	<code>String</code>	(Opcionalno) ID restorana vezanog uz obavijest
<code>dishId</code>	<code>String</code>	(Opcionalno) ID jela vezanog uz obavijest
<code>pollId</code>	<code>String</code>	(Opcionalno) ID ankete vezane uz obavijest
<code>attachment</code>	<code>Object</code>	Privitak: <code>{ name: String, url: String, type: String, size: String }</code>

polls

polls: Kolekcija za ankete koje kreiraju menze.

Atribut	Tip podatka	Opis
<code>_id (PK)</code>	<code>ObjectId</code>	Jedinstveni identifikator ankete
<code>title</code>	<code>String</code>	Naslov ankete
<code>questions</code>	<code>Array</code>	Niz pitanja (tekstualnih stringova)
<code>createdBy</code>	<code>String</code>	ID korisnika koji je kreirao anketu
<code>restaurantId</code>	<code>ObjectId</code>	Referenca na <code>restaurants</code>

answers

answers: Kolekcija za pohranu odgovora studenata na ankete.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator odgovora
<code>userId</code>	<code>String</code>	ID studenta koji odgovara
<code>pollId</code>	<code>String</code>	ID ankete na koju se odgovara
<code>answers</code>	<code>Array</code>	Niz odgovora: <code>[{ question: String, rating: Number }]</code>

readReceipts

readReceipts: Kolekcija za praćenje tko je pročitao koju broadcast obavijest.

Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator potvrde
<code>userId</code>	<code>String</code>	ID korisnika
<code>notificationId</code>	<code>String</code>	ID obavijesti
<code>readAt</code>	<code>Date</code>	Vrijeme kada je obavijest pročitana

allergens

allergens: Šifarnik alergena.

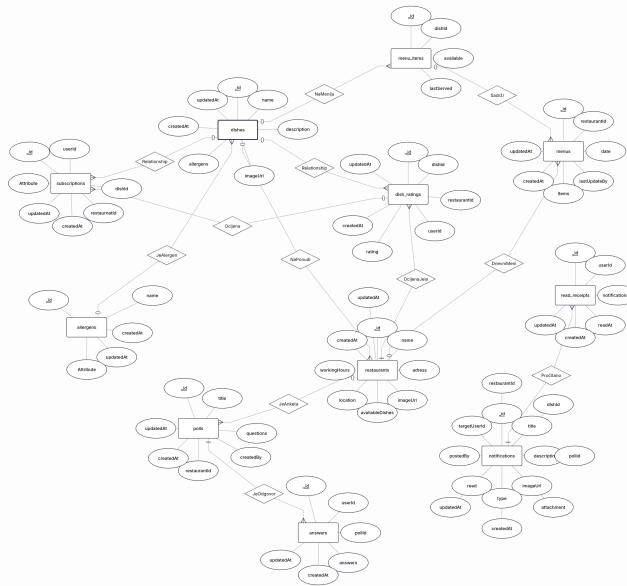
Atribut	Tip podatka	Opis
<code>_id</code> (PK)	<code>ObjectId</code>	Jedinstveni identifikator alergena
<code>name</code>	<code>String</code>	Naziv alergena (jedinstven)

Relacijski dijagram baze podataka

 Relacijski dijagram baze podataka

Slika 4.2: Shema MongoDB baze podataka

ER dijagram baze podataka

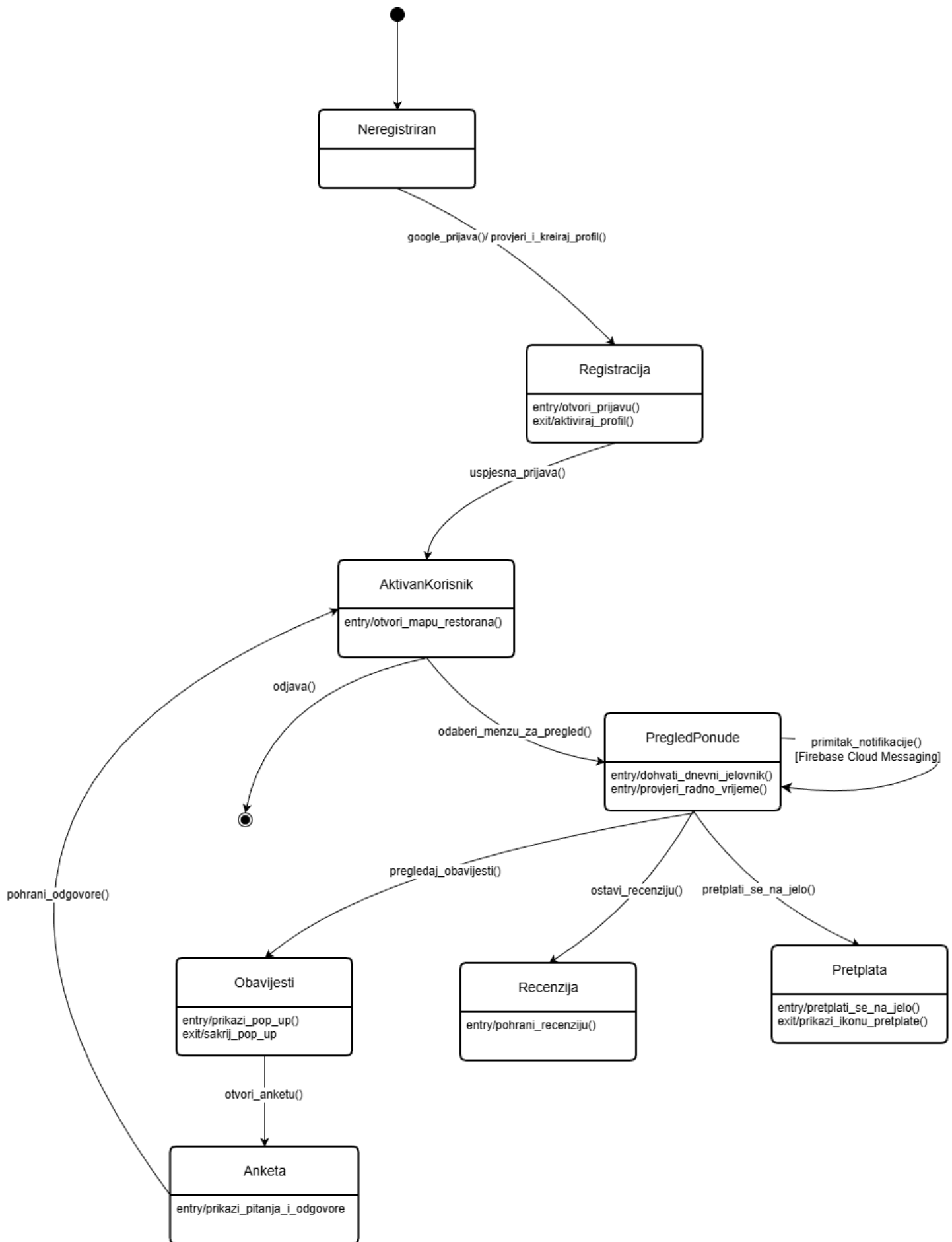


Slika 4.3: Shema MongoDB baze podataka

Dijagram razreda

Dijagram razreda prikazuje ključne modele (entitete) unutar aplikacije i njihove međusobne odnose. Svaki razred u dijagramu preslikava se u jedan dokument unutar odgovarajuće kolekcije u MongoDB bazi.

- **User:** Predstavlja korisnika sustava, bilo studenta ili zaposlenika.
- **Restaurant:** Predstavlja studentsku menzu.
- **Dish:** Predstavlja pojedinačno jelo.
- **Menu:** Predstavlja dnevni jelovnik za određenu menzu.
- **Review:** Predstavlja recenziju koju je korisnik ostavio za neko jelo.
- **DishSubscription:** Predstavlja grupirane pretplate na par jelo-menza.
- **Announcement:** Predstavlja obavijest vezanu za menzu.

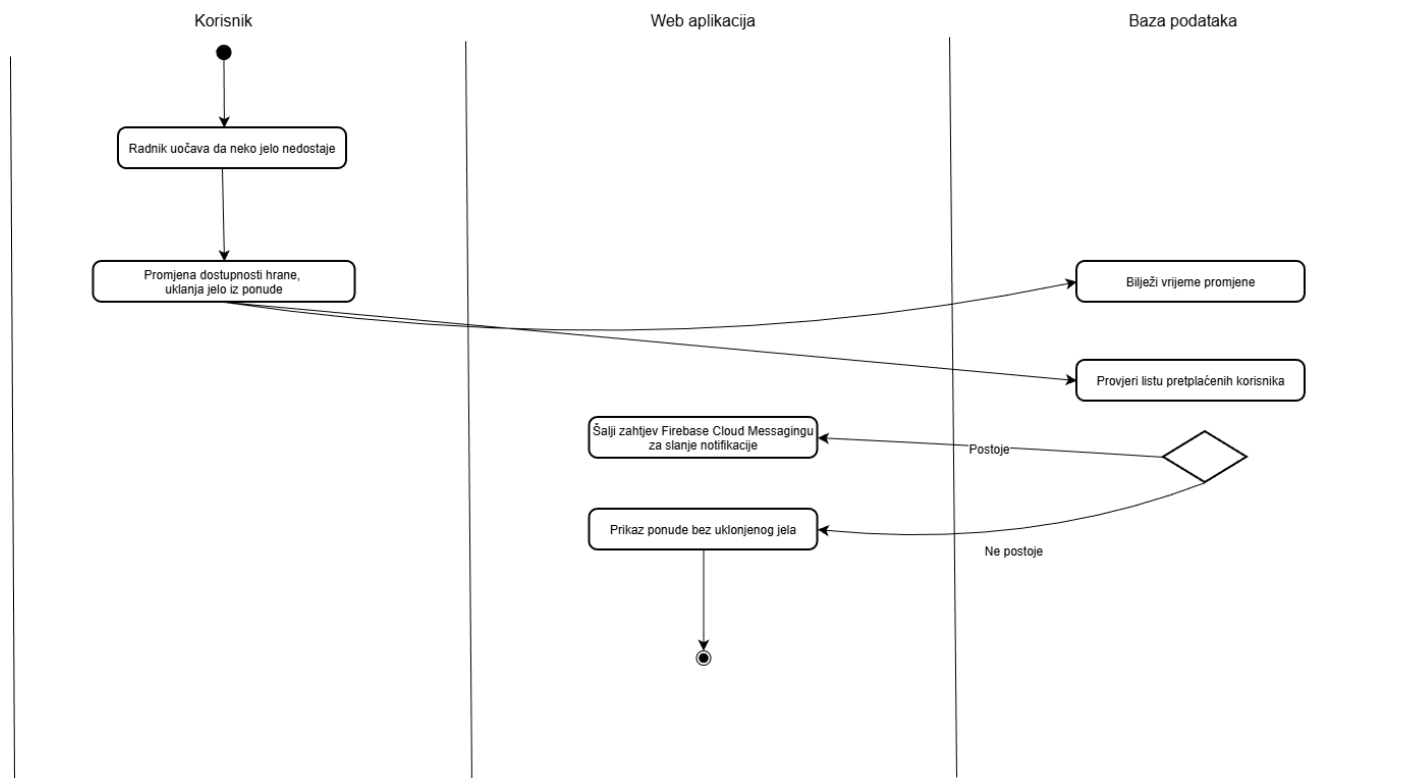


Slika 4.5: Dijagram stanja korisnika u ulozi studenta

Ovaj dijagram stanja prikazuje životni ciklus korisnika (studenta) koji započinje u stanju Neregistriran, iz kojeg se putem Google prijave pokreće proces provjere te kreiranja profila u bazi podataka ukoliko taj profil prijašnje nije postojao. Nakon uspješne autentifikacije, korisnik prelazi u stanje Pocetna gdje mu

se otvara mapa restorana, a daljnjim odabirom pojedine menze odlazi u stanje PregledPonude koje služi kao uvid u dostupna jela, radno vrijeme te primanje notifikacija o statusu hrane. Iz ovog središnjeg stanja student može interaktivno prelaziti u procese Pretplata za praćenje omiljenih jela, Recenzija za ocjenjivanje pojedinog jela ili Anketa za povremeno slanje povratnih informacija, dok se cijeli ciklus završava odjavom koja korisnika vraća u početni status.

UML dijagram aktivnosti



Slika 4.6: Dijagram aktivnosti

Ovaj dijagram aktivnosti vizualizira proces sustava koji osigurava informacije o dostupnosti jela u stvarnom vremenu. Proces je strukturiran kroz tri sekcije koje prikazuju interakciju radnika menze, web-aplikacije te baze podataka. Ključni trenutak procesa je grananje nakon upisa promjene u bazu podataka. Sustav istovremeno bilježi točno vrijeme promjene statusa jela i provjerava postojanje pretplaćenih korisnika radi slanja push notifikacija putem Firebase servisa. Ovakav dizajn minimizira kašnjenje informacija i osigurava da studenti prime obavijest odmah nakon što radnik evidentira nestanak jela, čime se direktno ispunjava primarni cilj aplikacije.

5. Arhitektura sustava

Frontend

Frontend je izgrađen koristeći **Next.js** framework. Organiziran je hijerarhijski:

- **Ulazna točka:** Aplikacija počinje s App (layout.tsx) koja prosljeđuje zahtjeve na **Ruter** (nextjs-app-router).
- **Glavni moduli (Role):** Ruter usmjerava korisnike na različite module ovisno o njihovoj ulozi ili statusu:
 - **Javni/Auth dijelovi:** main (početna), login, auth redirect, unassigned (nedodijeljeni korisnici).
 - **Admin:** Ima pristup upravljanju korisničkim računima (admin/accounts) i restoranima (admin/restaurants). Ovi moduli se dalje granaju na komponente za kreiranje (create) i ažuriranje (update).
 - **Manager (Voditelj):** Upravlja obavijestima (announcements), radnim vremenom (workingHours), jelovnicima (menu), anketama (poll) i statistikom (stats).
 - **Worker (Radnik):** Ima pristup dijelu funkcionalnosti managera, specifično manager/menu (prikazano isprekidanom strelicom).
 - **Student:** Ima pristup karti (map), anketama, pregledu restorana i obavijestima (notification).

Backend

Backend služi kao posrednik između Frontenda i baze/vanjskih servisa.

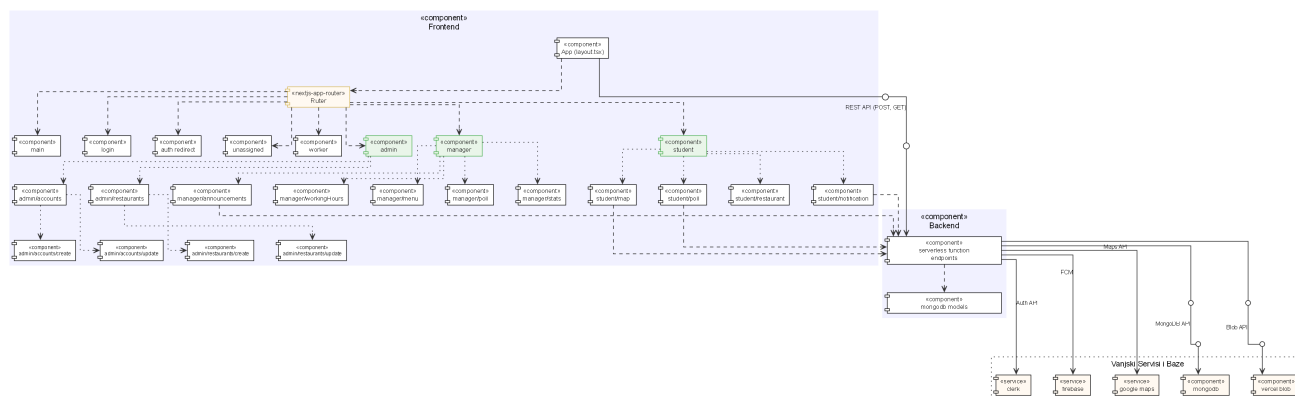
- **Serverless Endpoints:** Za backend se koristi serverless arhitektura hostana na Vercel platformi.
- **Komunikacija:** Frontend komunicira s Backendom putem REST API-ja (koristeći POST i GET metode).
- **Modeli podataka:** Backend koristi MongoDB modele za definiranje strukture podataka prije komunikacije s bazom.

Vanjski Servisi i Baze

Dish-detective se oslanja na niz vanjskih SaaS (Software as a Service) rješenja:

- **Clerk:** Auth API – Servis za autentifikaciju i upravljanje korisnicima (Login/Signup).
- **Firebase:** FCM (Firebase Cloud Messaging) – Koristi se za slanje push notifikacija studentima.
- **Google Maps:** Maps API – Za prikaz lokacija restorana na karti (koristi ga student/map preko backenda).
- **MongoDB:** Baza podataka (MongoDB API) – Glavno spremište podataka.

- Slika 5.1:** Detaljni hijerarhijski dijagram komponentata



Sustav Dish Detective implementiran je korištenjem moderne cloud-native i serverless arhitekture, čime se osiguravaju visoka dostupnost, automatsko skaliranje i minimalni operativni troškovi.

5.2.1. Korisnički uređaj (Client Side)

- pametne telefone
- tablete
- osobna računalna

- Web preglednik (Chrome, Safari, Firefox)

- Next.js klijentska aplikacija (frontend komponente)

- renderiranje korisničkog sučelja
- upravljanje lokalnim stanjem aplikacije
- interakciju s korisnikom

5.2.2. Cloud platforma (Vercel)

Središnji sloj aplikacijske logike smješten je na Vercel infrastrukturi, koja omogućuje distribuirano i skalabilno izvođenje koda.

Izvršno okruženje

- Next.js Serverless Functions (Node.js runtime)
- Funkcije se pokreću na zahtjev, bez stalno aktivnog poslužitelja

Backend endpoints

- Next.js API rute koje služe kao REST sučelje za klijentsku aplikaciju

Ovaj sloj obrađuje:

- poslovnu logiku
- autentifikaciju korisnika
- orkestraciju poziva prema vanjskim servisima

5.2.3. Vanjska upravljana infrastruktura (Managed Services)

Sustav se oslanja na veći broj vanjskih servisa:

- **MongoDB Atlas**

Potpuno upravljana baza podataka s automatskim sigurnosnim kopijama i distribuiranom pohranom podataka.

- **Clerk Auth**

Infrastruktura za upravljanje identitetom i korisničkim sesijama.

- **Firebase (FCM)**

Googleov servis za slanje push obavijesti mobilnim i web klijentima u stvarnom vremenu.

- **Google Maps Platform**

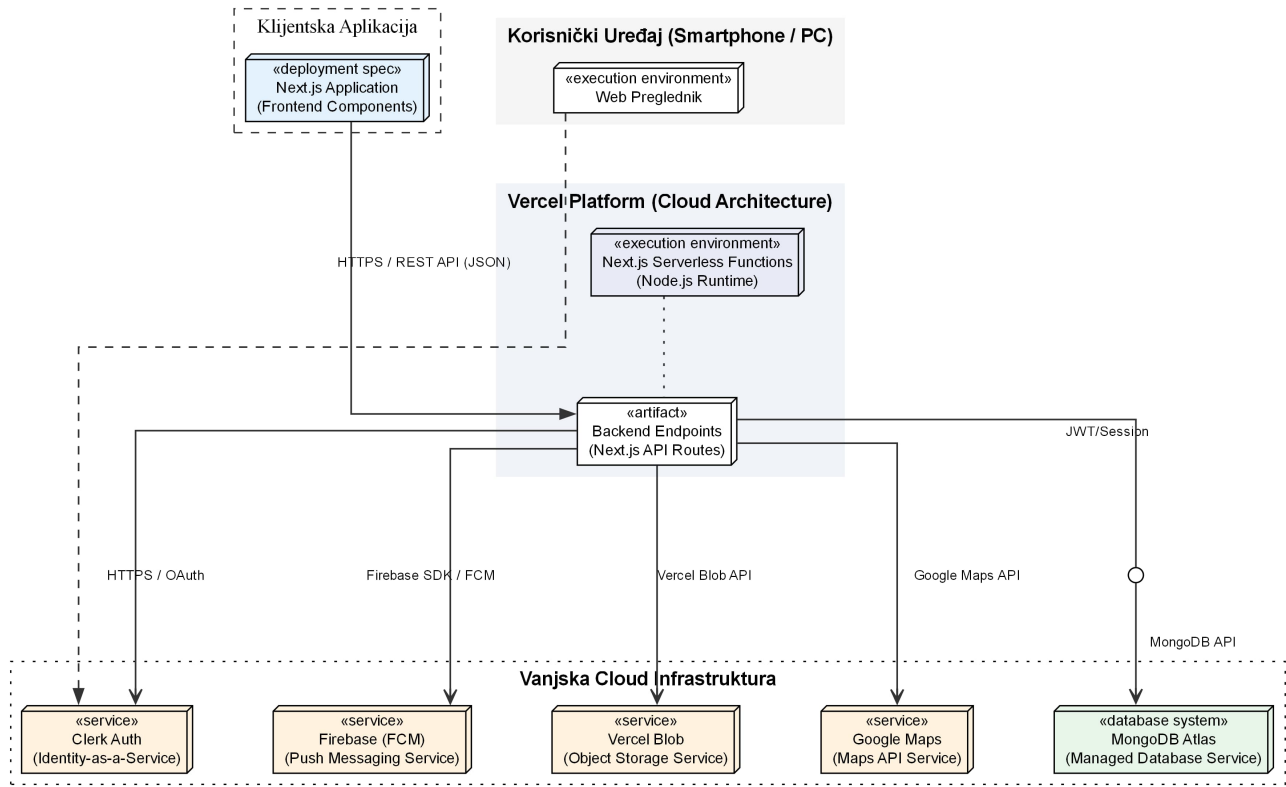
Servis za:

- geokodiranje
- prikaz karata
- izračun udaljenosti restorana

- **Vercel Blob**

Distribuirana objektna pohrana za multimedijski sadržaj (npr. fotografije jelovnika).

Slika 5.2: Dijagram razmjesta



Ispitivanje komponenti

1. Komponenta: Preglednik korisničkih profila radnika

Test 1: Uspješno dohvaćanje imena korisnika

1. **Opis testa:** Provjerava funkcionalnost kada je korisnik ispravno autenticiran i ima postavljeno ime u bazi podataka.

2. Ulazni podaci:

- Testni userId: "clerk_user_123"
- Testni podaci korisnika: { firstName: "John", lastName: "Doe" }

3. Očekivani rezultati:

- Funkcija vraća objekt: { success: true, firstName: "John", error: null }
- Pozvane su funkcije `auth()` i `clerkClient()`.

- `getUser` je pozvan s točnim ID-em.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Postaviti simulaciju autentifikacije da vrati važeći identifikator korisnika.
- Konfigurirati testni klijent da vrati korisnika s imenom "John".
- Pozvati funkciju `getCurrentUserFirstName`.
- Usporediti dobivene rezultate (status uspjeha, ime, grešku) s očekivanim vrijednostima.
- Potvrditi da su pozvani vanjski servisi s točnim parametrima.

Test 2: Provjera neautenticiranog korisnika

1. **Opis testa:** Provjerava sigurnosnu logiku kada korisnik nije prijavljen (nema sesije).

2. **Ulazni podaci:**

- Rezultat testne autentifikacije: `{ userId: null }`

3. **Očekivani rezultati:**

- Funkcija vraća objekt: `{ success: false, firstName: null, error: "Not authenticated" }`
- `clerkClient` se **ne** poziva.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Postaviti simulaciju autentifikacije tako da vrati null (simulacija neprijavljenog korisnika).
- Pozvati funkciju `getCurrentUserFirstName`.
- Provjeriti vraća li funkcija točnu poruku o grešci ("Not authenticated").
- Potvrditi da funkcija nije pokušala kontaktirati vanjski servis za dohvat korisnika.

Test 3: Rukovanje greškama Clerk API-ja

1. **Opis testa:** Provjerava otpornost funkcije na greške koje dolaze od strane vanjskog servisa (Clerk) prilikom dohvaćanja podataka o korisniku.

2. **Ulazni podaci:**

- Testni userId: "clerk_user_789"
- Simulirana greška: Error("Clerk API error: User not found")

3. Očekivani rezultati:

- Funkcija vraća objekt: { success: false, firstName: null, error: "Failed to fetch user information" }

4. Dobiveni rezultati: Prolaz ispitivanja (Pass).

5. Opis postupka provođenja ispitivanja:

- Simulirati uspješnu prijavu korisnika.
- Konfigurirati testni klijent tako da baci iznimku (grešku) prilikom poziva metode za dohvat korisnika.
- Pozvati funkciju getCurrentUserFirstName.
- Provjeriti je li try-catch blok uhvatio grešku.
- Potvrditi da funkcija vraća generičku poruku o neuspjehu umjesto rušenja aplikacije.

Test 4: Rukovanje greškama autentifikacijskog servisa

1. Opis testa: Provjerava ponašanje kada sam servis za autentifikaciju (auth()) zakaže i baci grešku.

2. Ulazni podaci:

- Simulirana greška na auth(): Error("Authentication service unavailable")

3. Očekivani rezultati:

- Funkcija vraća objekt: { success: false, firstName: null, error: "Failed to fetch user information" }
- clerkClient se ne poziva.

4. Dobiveni rezultati: Prolaz ispitivanja (Pass).

5. Opis postupka provođenja ispitivanja:

- Postaviti testnu funkciju autentifikacije da baci grešku odmah pri pozivu.
- Pozvati funkciju getCurrentUserFirstName.
- Provjeriti upravlja li funkcija ispravno iznimkom.

- Potvrditi da je vraćena predviđena poruka o grešci.
- Osigurati da se nije pokušalo pristupiti podacima korisnika jer je prijava neuspješna.

Test 5: Podrška za posebne znakove u imenu

1. **Opis testa:** Provjerava ispravno procesiranje imena koja sadrže dijakritičke znakove ili druge posebne karaktere (UTF-8 podrška).

2. **Ulazni podaci:**

- Testni userId: "clerk_user_special"
- Ime: "François-René"

3. **Očekivani rezultati:**

- Funkcija vraća: { success: true, firstName: "François-René", error: null }

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Postaviti testne podatke koji sadrže ime s posebnim znakovima.
- Simulirati povrat tih podataka kroz testni klijent.
- Pozvati funkciju getCurrentUserFirstName.
- Provjeriti je li vraćeni string identičan ulaznom (bez promjena u kodiranju znakova).

Test 6: Podrška za vrlo dugačka imena

1. **Opis testa:** Provjerava granični slučaj s ekstremno dugačkim imenom kako bi se osiguralo da nema nepredviđenog skraćivanja ili grešaka u memoriji/bufferu.

2. **Ulazni podaci:**

- Testni userId: "clerk_user_long"
- Ime: Tekst od 100 znakova "A" ("AAAA...").

3. **Očekivani rezultati:**

- Funkcija vraća ime duljine točno 100 znakova.
- success je true.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Generirati tekstualni niz od 100 znakova.
- Postaviti taj niz kao ime korisnika u testnom okruženju.
- Pozvati funkciju `getCurrentUserFirstName`.
- Provjeriti je li vraćeno ime identično ulaznom nizu.
- Potvrditi da je duljina vraćenog imena točno 100 znakova.

Izvorni kod testova 1. komponente

Izvorni kod testova nalazi se u <src/app/admin/actions.test.ts>

2. Komponenta: Forma za kreaciju, izmjenu i brisanje restorana

Test 1: Cjelokupni životni ciklus restorana (CRUD operacije)

1. **Opis testa:** Ovo je sveobuhvatan test koji provjerava cijeli tok upravljanja restoranima: stvaranje više restorana, izmjenu njihovih podataka, dohvaćanje sortirane liste i konačno brisanje svih zaposlenih zapisa.

2. **Ulazni podaci:**

- Podaci za 5 različitih restorana (npr. "Pizza Paradise", "Sushi House") s različitim radnim vremenima i lokacijama.
- Podaci za ažuriranje: nova adresa za prvi restoran, novo ime i slika za treći, novo radno vrijeme za peti.

3. **Očekivani rezultati:**

- Uspješno kreiranje svih 5 restorana.
- Uspješno ažuriranje odabranih restorana s točno primijenjenim izmjenama.
- Dohvaćanje liste vraća svih 5 restorana sortiranih abecedno po imenu.
- Brisanje svakog restorana prolazi uspješno.

- Konačna provjera pokazuje da je baza prazna (0 restorana).

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Pokrenuti petlju za stvaranje 5 restorana u testnoj bazi podataka i spremiti njihove ID-ove.
- Izvršiti funkciju ažuriranja nad tri specifična restorana s različitim parametrima.
- Dohvatiti sve restorane i provjeriti jesu li sortirani po imenu.
- Validirati specifična polja (npr. je li adresa prvog restorana promijenjena, je li slika trećeg ažurirana).
- Pokrenuti brisanje za svaki ID pojedinačno.
- Potvrditi da funkcija za dohvat sada vraća praznu listu.

Test 2: Rukovanje pogreškama (neispravni ID-ovi i validacija)

1. **Opis testa:** Provjerava otpornost sustava na neispravne ulazne podatke, uključujući nepostojeće ID-ove i nedostajuća obavezna polja.

2. **Ulazni podaci:**

- Neispravan ID za ažuriranje: "invalid-id".
- Nepostojeći (generirani) ID za brisanje.
- Pokušaj stvaranja restorana s praznim imenom i nepotpunim podacima.

3. **Očekivani rezultati:**

- Sve operacije vraćaju status neuspjeha (success: false).
- Vraćaju se odgovarajuće poruke o greškama ("Invalid", "not found", definisane greške validacije).

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Pozvati funkciju ažuriranja s namjerno neispravnim formatom ID-a.
- Generirati validan MongoDB ID koji ne postoji u bazi i pokušati brisanje.
- Pokušati kreirati restoran s objektom koji ne zadovoljava shemu (prazan string za ime).

- Za svaki slučaj provjeriti je li sustav vratio očekivanu oznaku greške umjesto rušenja ili neispravnog uspjeha.

Test 3: Neovisno ažuriranje više restorana

1. **Opis testa:** Potvrđuje da izmjene na jednom restoranu ne utječu na podatke drugog restorana.

2. **Ulazni podaci:**

- Restoran A i Restoran B.
- Ažuriranje imena za Restoran A.
- Ažuriranje adrese za Restoran B.

3. **Očekivani rezultati:**

- Restoran A ima novo ime, stara adresa ostaje.
- Restoran B ima staro ime, nova adresa je postavljena.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Kreirati dva zasebna restorana u testnoj bazi.
- Izvršiti update na prvom, pa na drugom s različitim poljima.
- Dohvatiti oba zapisa.
- Provjeriti križnu kontaminaciju podataka (osigurati da promjena na A nije utjecala na B i obrnuto).

Test 4: Podrška za složena radna vremena

1. **Opis testa:** Provjerava može li sustav podnijeti velike nizove podataka za radno vrijeme (npr. više smjena svaki dan u tjednu).

2. **Ulazni podaci:**

- Restoran s definiranih 7 dana, gdje svaki dan ima 3 različite smjene.

3. **Očekivani rezultati:**

- Kreiranje uspješno.
- Dohvaćeni restoran ima točno 7 unosa u polju workingHours.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Generirati objekt s kompleksnim rasporedom radnog vremena.
- Kreirati restoran s tim podacima.
- Dohvatiti restoran i provjeriti duljinu niza radnog vremena.

Test 5: Konkurentne operacije

1. **Opis testa:** Provjerava ponašanje sustava kada se više zahtjeva za stvaranjem restorana šalje istovremeno.

2. **Ulazni podaci:**

- 3 poziva za stvaranje restorana upućena paralelno (Promise.all).

3. **Očekivani rezultati:**

- Svi pozivi završavaju uspješno.
- U bazi se nalaze 3 restorana.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Pripremiti niz Promise-a za kreiranje tri različita restorana.
- Izvršiti ih paralelno.
- Provjeriti jesu li svi rezultati uspješni.
- Provjeriti ukupan broj zapisa u bazi.

Izvorni kod testova 2. komponente

Izvorni kod testova nalazi se u <src/app/admin/restaurants/actions.test.ts>

3. Komponenta: Forma za kreaciju i brisanje profila radnika

Test 1: Dohvaćanje svih zaposlenika s detaljima

1. **Opis testa:** Provjerava ispravno dohvaćanje liste zaposlenika i povezivanje njihovih metapodataka s nazivima restorana iz baze podataka.

2. **Ulazni podaci:**

- Testni restoran ID: "507f1f77bcf86cd799439011", Naziv: "Test Restaurant".
- Dva zaposlenika: jedan s ulogom "worker", drugi s ulogom "manager", oba povezana s navedenim restoranom.

3. **Očekivani rezultati:**

- Funkcija vraća objekt { success: true } s listom od dva zaposlenika.
- Svaki zaposlenik u listi sadrži ispravno mapirano ime restorana ("Test Restaurant").

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Postaviti simulaciju baze podataka da za traženi ID vrati objekt s nazivom restorana.
- Pripremiti listu testnih korisnika (radnik i voditelj) s odgovarajućim metapodacima.
- Simulirati povrat te liste od strane vanjskog servisa.
- Pozvati funkciju getAllEmployees.
- Provjeriti sadrži li rezultat točan broj zaposlenika i jesu li podaci (ime, uloga, naziv restorana) ispravni.

Test 2: Rukovanje slučajem nepostojećeg restorana

1. **Opis testa:** Provjerava otpornost sustava kada zaposlenik ima referencu na restoran koji više ne postoji u bazi podataka.

2. **Ulazni podaci:**

- Zaposlenik s restaurantId koji ne postoji u bazi.
- Upit prema bazi za taj ID vraća null.

3. **Očekivani rezultati:**

- Funkcija vraća uspješan rezultat.

- Za navedenog zaposlenika polje restaurantName postavljeno je na "Nije pridodijeljen".

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Simulirati bazu podataka tako da vrati null (restoran nije pronađen).
- Pripremiti testnog korisnika s referencom na taj nepostojeći restoran.
- Pozvati funkciju getAllEmployees.
- Provjeriti je li sustav ispravno dodijelio rezervnu vrijednost za naziv restorana umjesto rušenja aplikacije.

Test 3: Sigurnosna provjera ovlasti administratora (dohvat)

1. **Opis testa:** Provjerava je li onemogućen pristup podacima korisnicima koji su prijavljeni, ali nemaju administratorsku ulogu (npr. studenti).

2. **Ulazni podaci:**

- Prijavljeni korisnik s ulogom "student".

3. **Očekivani rezultati:**

- Funkcija vraća: { success: false, error: "Only admins can view employee accounts" }.

4. **Dobiveni rezultati:** Prolaz ispitivanja (Pass).

5. **Opis postupka provođenja ispitivanja:**

- Simulirati prijavu korisnika koji ima ulogu "student".
- Pozvati funkciju getAllEmployees.
- Provjeriti vraća li funkcija specifičnu poruku o zabrani pristupa za ne-administratore.

Test 4: Uspješno brisanje zaposlenika

1. **Opis testa:** Provjerava funkcionalnost brisanja korisničkog računa zaposlenika.

2. **Ulazni podaci:**

- ID korisnika za brisanje: "clerk_delete_test".
- Ciljani korisnik ima ulogu "worker".

3. Očekivani rezultati:

- Funkcija vraća: { success: true }.
- Pozvana je metoda deleteUser s točnim ID-em.

4. Dobiveni rezultati: Prolaz ispitivanja (Pass).

5. Opis postupka provođenja ispitivanja:

- Simulirati da vanjski servis pronađe korisnika s ulogom "worker".
- Simulirati uspješno izvršavanje naredbe za brisanje.
- Pozvati funkciju deleteEmployee s ciljanim ID-em.
- Provjeriti uspješnost rezultata i potvrditi poziv metode za brisanje.

Test 5: Sprječavanje brisanja administratora

1. **Opis testa:** Provjerava sigurnosnu logiku koja sprječava da administrator obriše drugog administratora (ili samog sebe) putem ove funkcije.

2. Ulazni podaci:

- ID ciljanog korisnika: "clerk_admin_delete_test".
- Ciljani korisnik ima ulogu "admin".

3. Očekivani rezultati:

- Funkcija vraća: { success: false, error: "Can only delete manager, worker, or unassigned accounts" }.

4. Dobiveni rezultati: Prolaz ispitivanja (Pass).

5. Opis postupka provođenja ispitivanja:

- Konfigurirati testni servis da za traženi ID vrati korisnika s ulogom "admin".
- Pozvati funkciju deleteEmployee.
- Provjeriti je li operacija odbijena uz odgovarajuću poruku o grešci.

Test 6: Sigurnosna provjera ovlasti administratora (brisanje)

1. **Opis testa:** Provjerava je li onemogućeno brisanje zaposlenika korisnicima koji nemaju administratorska prava.

2. Ulazni podaci:

- Prijavljeni korisnik s ulogom "student".

3. Očekivani rezultati:

- Funkcija vraća: { success: false, error: "Only admins can delete employee accounts" }.

4. Dobiveni rezultati: Prolaz ispitivanja (Pass).

5. Opis postupka provođenja ispitivanja:

- Simulirati prijavu korisnika s ulogom "student".
- Pozvati funkciju deleteEmployee.
- Provjeriti sprječava li sustav brisanje i vraća li poruku o nedostatku ovlasti.

Izvorni kod testova 3. komponente

Izvorni kod testova nalazi se na <src/app/admin/accounts/actions.test.ts>

Svi testovi automatski se izvršavaju pri svakom pull request (postignuto koristeći github actions).

 playwright res

Ispitivanje sustava

Test: admin dish creation

1. Ulazi:

- Podaci za prijavu administratora (korisničko ime: 'admin', lozinka: 'admin').
- Podaci o jelu: opis 'test jelo', lažna .png slika (kreirana iz međuspremnika), alergen 'Jaje', naziv 'Testno jelo'.

2. Koraci ispitivanja:

1. Navigacija na URL <http://employee.localhost:3000/login/employee>.
2. Unos korisničkog imena i lozinke te klik na 'Prijavi se'.
3. Odabir sekcije 'Jela'.
4. Klik na gumb 'add dish'.

5. Unos opisa jela.

6. Učitavanje slike izravnim umetanjem datoteke (buffer) u input polje.

7. Odabir alergena iz padajućeg izbornika.

8. Unos naziva jela.

9. Klik na gumb 'Kreiraj jelo'.

10. Otvaranje korisničkog izbornika i klik na 'Sign out'.

3. **Očekivani izlazi:** Sustav uspješno kreira novo jelo i odjavljuje administratora bez grešaka.

4. **Dobiveni izlaz:** test je prošao uspješno

 kreacija jela

 jelo kreirano

Test: admin dish creation of already existing dish

1. Ulazi:

- Podaci za prijavu administratora (admin/admin).
- Podaci o jelu koji su identični već postojećem jelu (naziv 'Testno jelo').

2. Koraci ispitivanja:

1. Navigacija na URL <http://employee.localhost:3000/login/employee> i prijava u sustav.
2. Odabir sekcije 'Jela' i klik na 'add dish'.
3. Ispunjavanje forme za jelo (opis, slika, alergen).
4. Unos naziva jela koje već postoji u bazi.
5. Klik na gumb 'Kreiraj jelo'.
6. Provjera pojavljuje li se poruka o dupliciranom nazivu.

3. **Očekivani izlazi:** Prikaz poruke upozorenja "A dish with this name already".

4. **Dobiveni izlaz:** test je prošao uspješno

 jelo postoji

Test: wrong login for worker

1. Ulazi:

- Podaci za prijavu radnika s pogrešnom lozinkom (korisničko ime: 'worker', lozinka: 'test').

2. Koraci ispitivanja:

1. Navigacija na URL <http://employee.localhost:3000/login/employee>.
2. Unos korisničkog imena 'worker'.
3. Unos pogrešne lozinke 'test'.
4. Klik na gumb 'Prijavi se'.
5. Provjera pojavljuje li se poruka o neuspješnoj prijavi.

3. **Očekivani izlazi:** Prikaz poruke "Neispravno korisničko ime ili".

4. **Dobiveni izlaz:** test je prošao uspješno

 krivi password

Test: manual redirect test

1. Ulazi:

- Ispravni podaci za prijavu radnika (korisničko ime: 'worker', lozinka: 'worker').
- Ciljani URL '/admin'.

2. Koraci ispitivanja:

1. Navigacija na URL <http://employee.localhost:3000/login/employee>.
2. Unos ispravnih podataka i prijava u sustav.
3. Ručna navigacija (izravan odlazak) na URL <http://employee.localhost:3000/admin>.

3. **Očekivani izlazi:** Preglednik odbija navigaciju na admin rutu i vraća korisnika na <http://employee.localhost:3000/worker>.

4. **Dobiveni izlaz:** test je prošao uspješno

Svi testovi automatski se izvršavaju pri svakom pull request (postignuto koristeći github actions).

 playwright res

Korištene tehnologije i alati

1. Programski jezici:

- **TypeScript 5.0** korišten je za razvoj i klijentskog i serverskog dijela aplikacije, odabran zbog mogućnosti razvijanja frontend i backend dijela u istom programskom jeziku. Odabirom TypeScripta ostvaruje se mogućnost korištenja Next.js radnog okvira koji pojednostavljuje implementaciju aplikacije. Prednost nad JavaScriptom jest podržavanje tipova što omogućuje hvatanje grešaka tijekom pisanja koda.

2. Radni okviri i biblioteke:

- **React 19.2.2** odabran je zbog fleksibilnosti i mogućnosti ponovnog korištenja komponenti. Korištenje Reacta pojednostavljuje ostvarivanje dinamičnosti na klijentu.
- **Next.js 16.0.10** koristi se zbog pojednostavljene integracije backend API-a s klijentom te jednostavnog puštanja u pogon pomoću Vercel platforme. Njegova popularnost omogućila je brzo rješavanje problema na koje smo naišli.
- **Clerk 6.34.5** korišten je za autentifikaciju i autorizaciju korisnika te za spremanje podataka o korisniku. Clerk je odabran zbog provjerene poizdanosti u sigurnost podataka.
- **Mongoose 8.19.3** korišten je za pristup i validaciju podataka iz baze, a odabran je zbog native integracije s Next.js-om.

3. Baza podataka:

- **MongoDB 6.19.0** noSQL baza podataka odabrana je zbog jednostavne integracije s JavaScriptom te zbog svoje pouzdanosti i skalabilnosti.
- **Vercel Blob** odabran zbog native integracije s Vercelom te jednostavnosti pohranjivanja statičnih datoteka.

4. Razvojni alati:

- **VSCode 1.81.0** korišten je zbog svoje brzine, fleksibilnosti i odličnog ekosustava ekstenzija za TypeScript.
- **Github** korišten za verziranje koda, suradnju i komunikaciju na projektu.
- **Figma 125.11.6** za dizajn korisničkog sučelja aplikacije i planiranja rasporeda komponenti

5. Alati za ispitivanje:

- **Jest 30.2.0** korišten za ispitivanje individualnih komponenti i automatskog testiranja pri svakom pull requestu na Githubu.
- **Playwright 1.56.1** korišten za automatizirano end-to-end testiranje.

6. Alati za razmjestaj:

- **npm 11.6.2** korišten za upravljanje ovisnostima unutar projekta te za pokretanje testova.

7. Cloud platforma:

- **Vercel** odabran zbog jednostavnosti puštanja aplikacije u pogon i zbog svoje pouzdanosti i skalabilnosti

Ovaj dokument opisuje proces pokretanja aplikacije u lokalnom razvojnom okruženju (Development) i postavljanje na produkcijsku okolinu (Production) koristeći Vercel.

1. Lokalno pokretanje (Development)

Za pokretanje projekta na lokalnom računalu potrebno je imati instaliran [Node.js](#) i `npm`.

Koraci:

1. Kloniraj repozitorij:

```
git clone https://github.com/Dish-Detectors/dish-detective.git
cd dish-detective
```

2. Instaliraj zavisnosti:

```
npm install
```

3. Postavi varijable okruženja:

Kopiraj i zalijepi datoteku `.env.local.example` i preimenuj ju u `.env.local` u korijenskom direktoriju projekta. Popuni ju svojim API ključevima (vidi sekciju "Konfiguracija Servisa").

4. Pokreni razvojni server:

```
npm run dev
```

5. Otvori u pregledniku:

Aplikacija je dostupna na `http://localhost:3000`.

2. Konfiguracija vanjskih servisa

Prije nego aplikacija može ispravno raditi, potrebno je pribaviti ključeve sa sljedećih servisa:

Clerk (Autentifikacija)

1. Kreiraj aplikaciju na [Clerk Dashboardu](#).
2. U postavkama ("API Keys") preuzmi `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` i `CLERK_SECRET_KEY`.
3. **Važno:** U postavkama "Paths" provjeri da se URL-ovi za login poklapaju s onim u `.env` datoteci.

MongoDB Atlas (Baza podataka)

1. Kreiraj klaster na [MongoDB Atlasu](#).
2. Kreiraj bazu i korisnika (Database Access).
3. Dohvati "Connection String" (odaberi "Connect via Drivers") i zalijepi ga pod `MONGODB_URI`.

Google Cloud (Karte i Mjesta)

1. Otvori [Google Cloud Console](#).
2. Omogući **Maps JavaScript API** i **Places API (New)**.
3. Kreiraj Credentials (API Key) i dodaj ga pod `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`.

Firebase (Notifikacije)

1. Kreiraj projekt na [Firebase Console](#).
2. U "Project Settings" -> "General" kreiraj Web aplikaciju kako bi dobio konfiguraciju (apiKey, authDomain, projectId...).
3. U "Project Settings" -> "Service accounts" generiraj novi privatni ključ za Admin SDK.
 - *Napomena:* `FIREBASE_PRIVATE_KEY` mora biti unutar dvostrukih navodnika zbog novih redova.

Vercel Blob (Slike)

Ovaj servis se konfigurira direktno kroz Vercel sučelje (vidi sekciju Produkcija).

3. Varijable okruženja (Environment Variables)

Ovo je predložak `.env` datoteke koju treba popuniti:

```
# Clerk Auth
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test...
CLERK_SECRET_KEY=sk_test...
NEXT_PUBLIC_CLERK_SIGN_IN_URL=/login/employee

# MongoDB Database
MONGODB_URI=mongodb+srv://<username>:<password>@...

# Vercel Blob Storage (Automatski se dodaje kod integracije na Vercelu)
BLOB_READ_WRITE_TOKEN=

# Google Maps Platform
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=AIza...

# Firebase Cloud Messaging & Admin
FIREBASE_PROJECT_ID=dish-detective-...
FIREBASE_CLIENT_EMAIL=...
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n..." # Obavezno navoditi

# Firebase Client Config
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=...
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=...
NEXT_PUBLIC_FIREBASE_APP_ID=...
NEXT_PUBLIC_FIREBASE_VAPID_KEY=...
```

4. Produkcija (Vercel)

Aplikacija se nalazi na Vercelu. Za postavljanje ("deploy") nove instance ili ažuriranje:

1. Povezivanje repozitorija:

- Na [Vercel Dashboardu](#) odaberi "Add New..." -> "Project".
- Importiraj `dish-detective` repozitorij s GitHuba.

2. Konfiguracija varijabli:

- U koraku konfiguracije, kopiraj sadržaj svoje lokalne `.env` datoteke u sekciju **Environment Variables**.

3. Vercel Blob Storage:

- Nakon inicijalnog deploya, u Vercel projektu idi na tab **Storage**.
- Odaberi "Connect Store" -> "Create New" -> "Blob".
- Ovo će automatski dodati `BLOB_READ_WRITE_TOKEN` u varijable okruženja.

4. Domena (Cloudflare):

- U Vercelu pod "Settings" -> "Domains" dodaj custom domenu.
- Na Cloudflare-u (DNS postavkama) dodaj `CNAME` ili `A` zapis prema uputama koje Vercel prikaže.

5. Deploy:

- Svaki `push` na `main` granu automatski će pokrenuti novi build i deploy na produkciju.

Cilj projekta Dish Detective bio je ostvariti responzivnu web aplikaciju koja će studentima poboljšati iskustvo u studentskim restoranima uvođenjem jelovnika koji se ažurira u stvarnom vremenu. Projektni je zadatak dovoljno opsežan i kompleksan te ga je trebalo podijeliti u logičke cjeline: izradu dokumentacije, frontend, backend te testiranje sustava. Tijekom prvog ciklusa, fokus je bio na preciznom planiranju funkcionalnosti aplikacije, izradi dokumentacije i izradi arhitekture, nakon čega je uslijedila implementacija ključnih funkcionalnosti poput Google autentifikacije i funkcionalnosti administratora, jednog od korisnika. Kasnije faze obuhvatile su razvoj specifičnih sučelja za radnike i voditelje menzi, studente koji su središte cijele aplikacije, notifikacije te Google Maps karte.

Tijekom ostvarenja projektnog zadatka naišli smo na nekoliko ključnih izazova:

1. Timski rad i povezivanje zadataka

Susreli smo se s izazovima integracije komponenti koje su međusobno usko povezane, a razvijale su ih različite osobe. Suočili smo se s povremenim neskladom između nekih dijelova frontenda i backenda kao što su drugačiji načini ostvarivanja funkcionalnosti iste komponente i nekonzistentnosti prilikom prikaza same stranice. Takvi su problemi lako rješivi kada cijeli tim surađuje te ima preciznu i jasnu komunikaciju. Navedeni problemi naglasili su važnost timskog rada koji je ključni dio ostvarenja ovog projektnog zadatka te je bez njega nemoguće pravilno napraviti aplikaciju. Napretkom projekta naš se timski rad usavršavao, a redovita komunikacija postala je ključna za održavanje konzistentnosti aplikacije.

2. Tehnički problemi

Kao i kod svakog kompleksnog projekta, pojavile su se neke tehničke poteškoće koje su zahtijevale upornost, jasnu komunikaciju i snalažljivost tima. Poseban izazov predstavljalo je ispravno implementiranje komponenti frontenda te njihovo međusobno usklađivanje. Također, integracija vanjskih servisa poput Clerk-a za autentifikaciju i Firebase-a za notifikacije iziskivala je dodatno vrijeme za učenje specifičnosti tih platformi kako bi se osigurala maksimalna sigurnost i pouzdanost sustava. Još jedna od prepreka s kojom smo se susreli bila je promjena načina na koji pristupamo pohranjivanju korisničkih računa zbog mogućnosti njihovog spremanja preko vanjskog servisa Clerka.

Projektni zadatak pružio nam je jedinstvenu priliku da uz mentorstvo i pomoć profesora, asistenta i demonstratora utvrdimo i upotrijebimo svoje teoretsko znanje u stvarnom okruženju. Nekim članovima tima ovo je prvi susret s projektom ove razine, što nam je pružilo priliku stjecanja novih vještina koje će biti presudne u našim budućim profesionalnim karijerama.

Tijekom izrade projekta naučili smo mnogo o suradnji, komunikaciji, modernim tehnologijama i razvitku aplikacije. Utvrdili smo svoja znanja planiranja, korištenja baze podataka, implementacije koristeći radna okruženja kao što su React i Next.js, te korištenje baze podataka poput MongoDB-a. Za još bržu i kvalitetniju realizaciju u budućnosti, smatramo da bi nam dodatno predznanje o radnim okruženjima, naprednim Git procesima (Gitflow), automatiziranom testiranju (CI/CD) te realnom postavljanju vremenskih ograničenja omogućilo još fluidniji razvojni proces.

Daljnji rad mogao bi se odnositi na nastavak razvijanja aplikacije koja bi mogla uključivati dodavanje novih funkcionalnosti poput sustava za praćenje gužva u pojedinim menzama i mogućnosti da aplikacija sama predlaže menze studentima na temelju njihovih recenzija jela.

Rev	Opis promjene	Autor	Datum
0.1.1	Dodao opis projekta	Andrea Marić	19.10.2025.
0.2.1	Popunili tablicu funkcionalnih zahtjeva	Andrea Marić, Luka Ražem	20.10.2025.
0.2.2	Dodao opis nefunkcionalnih zahtjeva i dionika	Matej Božac	22.10.2025.
0.2.3	Dodao aktore i njihove funkcionalne zahtjeve	Loren Čekada	22.10.2025.
0.2.4	Promijenio zahtjeve za administratora i voditelja menze	Luka Ražem	22.10.2025.
0.3.1	Dodali specifikacije zahtjeva sustava	Matea Maračić, Andrea Marić, Luka Ražem	23.10.2025.
0.3.2	Dodali UML dijagrame	Matea Maračić, Andrea Marić, Luka Ražem	28.10.2025.
0.3.3	Dodao dijagram obrasca uporabe	Luka Ražem	29.10.2025.
0.3.4	Popravila nekoliko UML dijagrama	Matea Maračić	29.10.2025.

Rev	Opis promjene	Autor	Datum
0.3.5	Dodao tablice aktivnosti grupe i provjere uključenosti ključnih funkcionalnosti u obrasce uporabe	Ivan Marfat	30.10.2025.
0.3.6	Dodao Clerk u dokumentaciju	Luka Ražem	2.11.2025.
0.4.1	Dodali tekst za arhitekturu i dizajn sustava	Andrea Marić, Luka Ražem	3.11.2025.
0.4.2	Dodali sekvencijske dijagrame	Andrea Marić, Luka Ražem	6.11.2025.
0.4.3	Dodao sekvencijske dijagrame	Matej Božac	6.11.2025.
0.4.4	Dodao MVC dijagram	Loren Čekada	11.11.2025.
0.4.5	Uredila opis projektnog zadatka	Matea Maračić	12.11.2025.
0.4.6	Dodala opise UML dijagrama	Matea Maračić	13.11.2025.
0.4.7	Uredila dokumentaciju, dodala UML dijagrame i opise sekvencijskih dijagrama	Matea Maračić	14.11.2025.
0.5.0	Dodao popis literature	Andrea Marić	14.11.2025.
0.5.1	Promijenila dijagrame	Matea Maračić	2.1.2026.
0.5.2	Dodao novi sastanak	Luka Ražem	12.1.2026.
0.5.3	Dodala dijagrame stanja i aktivnosti i opise	Matea Maračić	14.1.2026.
0.5.4	Popunili Tehnologije za implementaciju aplikacije	Andrea Marić, Matea Maračić	21.1.2026.
0.5.5	Dodao plan rada, dijagrame komponenti i razmještaja	Loren Čekada	22.1.2026.
0.6.0	Dodao upute za puštanje u pogon	Luka Ražem	23.1.2026.
0.7.0	Dodao testove i opis testova	Andrea Marić	23.1.2026.
0.7.1	Ispunio tablicu aktivnosti	Matej Božac	23.1.2026.

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langanieri, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, "Software engineering book", Department of Electrical and Computer Engineering, Rutgers University, [http://www.ece.rutgers.edu/~marsic/](http://www.ece.rutgers.edu/~marsic/books/SE) books/SE
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new>

Dnevnik sastajanja

1. sastanak

- Datum: 15. listopada 2025.
- Prisustvovali: L.Ražem, A.Marić, M.Maračić, L.Čekada, M.Božac, I.Marfat
- Teme sastanka:
 - uspostavljena Discord grupa svih članova i demonstratora
 - dogovor o provođenju projekta

2. sastanak

- Datum: 29. listopada 2025.
- Prisustvovali: L.Ražem, A.Marić, M.Maračić, L.Čekada, M.Božac, I.Marfat
- Teme sastanka:
 - razrada i dogovor oko projektnih odgovornosti
 - planiranje izrade

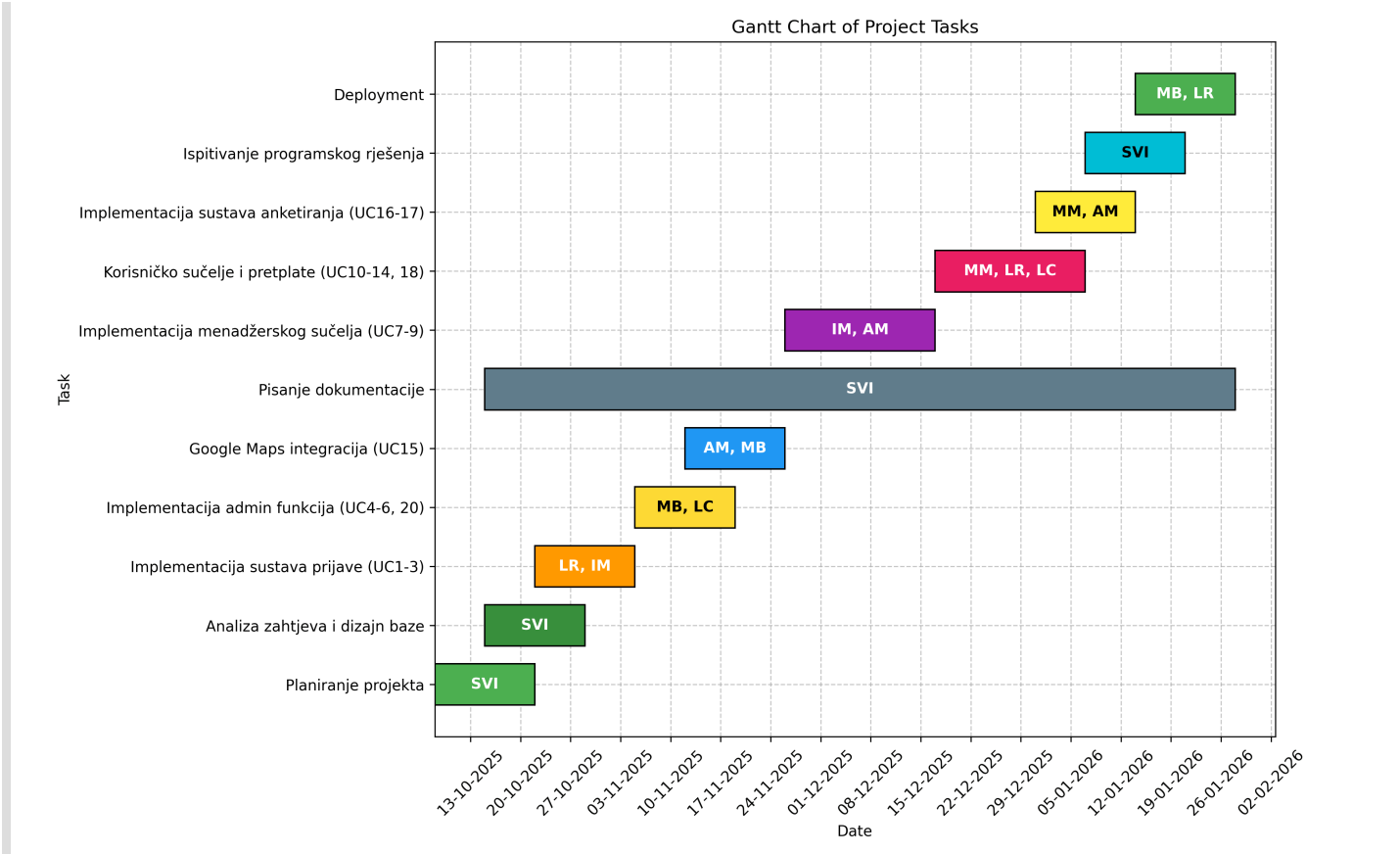
3. sastanak

- Datum: 9. studenog 2025.
- Prisustvovali: L.Ražem, A.Marić, M.Maračić, L.Čekada, M.Božac, I.Marfat
- Teme sastanka:
 - dogovor oko 1. sprinta

4. sastanak

- Datum: 12. siječnja 2026.
- Prisustvovali: L.Ražem, A.Marić, M.Maračić, L.Čekada, M.Božac, I.Marfat
- Teme sastanka:
 - dogovor oko 2. sprinta

Plan rada



Tablica aktivnosti

Kontinuirano osvježavanje

Komponenta	Luka Ražem	Andrea Marić	Matea Maračić	Loren Čekada	Matej Božac	Ivan Marfat
Upravljanje projektom	3	2	2			
Opis projektnog zadatka	3	2	3			
Funkcionalni zahtjevi	5	4	3		2	
Opis pojedinih obrazaca	2	2	2			
Dijagram obrazaca	3	4	6			5
Sekvencijski dijagrami	4				3	
Opis ostalih zahtjeva	3			15		
Arhitektura i dizajn sustava	5	5	3			
Baza podataka	4	4				
Dijagram razreda					5	
Dijagram stanja			4			

Komponenta	Luka Ražem	Andrea Marić	Matea Maračić	Loren Čekada	Matej Božac	Ivan Marfat
Dijagram aktivnosti			3			
Dijagram komponenti				5		
Dijagram razmještaja				3		
Rad na backendu	20	40				
Ogledni dizajn aplikacije			27			
Rad na frontendu	40	10		10	27	45
Izrada baze podataka	4	4				
Ispitivanje programskog rješenja	1	8		4	2	
Deployment	10					
Upute za puštanje u pogon	3					
Dnevnik sastajanja	1					
Zaključak i budući rad			2			
GitHub i dokumentacija	5	5	6	4	5	1
Izrada prezentacije projekta			5			

Dijagram pregleda promjena



Ključni izazovi i rješenja

Zaključno

Tijekom izrade aplikacije Dish-Detective iskoristili smo prijašnja znanja, ali i stekli brojna nova znanja o web developmentu, puštanju stranice u pogon, vođenju projekta, timskom radu i komunikaciji. Iako je projekt uspješno završen, susreli smo se s brojnim izazovima i problemima koji su od našeg tima zahtijevali komunikaciju, snalažljivost i sposobnosti rješavanja problema. S obzirom na to da je ovaj projekt mnogim studentima prvi susret s projektom na ovoj razini, ovo će znanje zasigurno biti korisno u pravom životu i poslovnoj karijeri.

Opis izazova

1. Držanje plana i rokovi

Imali smo mnogo odličnih ideja vezanih za komponente i funkcionalnosti naše stranice te smo morali odlučiti koje od njih ćemo moći implementirati, a koje ćemo morati izbaciti. Nakon toga, problem je bio uspjeti završiti implementaciju svega što smo odabrali u određenom vremenskom

periodu. Bilo je teško procijeniti koliko će nam vremena biti potrebno za pojedini dio projekta te smo morali optimizirati i prilagođavati se vremenskim rokovima i mogućnosti tima.

2. Tehnički problemi

Zbog toga što je Clerk sposoban upravljati cijelim profilom korisnika, morali smo promijeniti način pohrane korisničkih računa, odnosno ne spremati ih u bazu podataka te nam je to zadalo dodatnu prepreku. Također, morali smo odstupati od klasičnog MVC pristupa arhitekturi zbog Next.js-a. S obzirom na to da smo se susreli s ovim preprekama, morali smo nekoliko puta zastati i mijenjati način na koji ćemo pisati kod.

Rješenja

1. Realno planiranje i točnija pretpostavka o potrebnom vremenu za rješavanje pojedinih zadataka
2. Zastati, komunicirati i međusobno razraditi daljnji plan kako bi svi bili upoznati s promjenama u sustavu